

Bài giảng

Transfer Learning & Fine-tuning

Từ Feature Extraction đến LoRA & QLoRA

Module Deep Learning hiện đại

Giảng viên: Hà Minh Tuấn

Khoa Toán – Tin học, Trường Đại học Khoa học Tự nhiên (HCMUS)

Ngày 16 tháng 6 năm 2026

Tóm tắt nội dung

Tài liệu này là bài giảng đầy đủ cho module **Transfer Learning & Fine-tuning** trong chương trình Deep Learning dành cho sinh viên. Bài giảng được thiết kế theo thang Bloom (*Understand* → *Apply* → *Analyze* → *Create*), kết hợp giữa nền tảng toán học, trực giác (intuition), code thực hành, và các câu hỏi gợi mở nhằm phát triển tư duy phản biện. Nội dung trải dài từ những ý tưởng nền tảng (vì sao tái sử dụng tri thức lại hiệu quả) cho tới các kỹ thuật tiên tiến nhất hiện nay (PEFT, LoRA, QLoRA), cùng các tình huống ứng dụng thực tế trong Computer Vision và NLP.

Mục lục

1	Mục tiêu & Bản đồ học tập	3
1.1	Mục tiêu học tập (Learning Objectives)	3
1.2	Kiến thức tiên quyết (Prerequisites)	3
1.3	Bản đồ module	3
2	Nền tảng Lý thuyết của Transfer Learning	4
2.1	Warm-up: Một câu hỏi tưởng dễ	4
2.2	Động lực: ImageNet và “đứng trên vai người khổng lồ”	4
2.3	Định nghĩa hình thức	5
2.4	Phân biệt ba khái niệm trung tâm	6
2.5	Toán học đằng sau: vì sao biểu diễn lại tái sử dụng được	6
2.6	Khi miền lệch nhưng tác vụ không đổi: Domain Adaptation	7

3	Feature Extraction — Đào sâu & Thực hành	9
3.1	Câu hỏi mở đầu	9
3.2	Vì sao tầng đầu học low-level, tầng sau học high-level?	9
3.3	Cơ chế Freezing: <code>requires_grad = False</code>	9
3.4	Code mẫu: Feature Extraction với ResNet-50	10
3.5	Giới thiệu Lab 1	11
3.6	Visualization: nhìn vào không gian đặc trưng	11
4	Fine-tuning & Các Thực hành Tốt nhất	12
4.1	Câu hỏi mở đầu	12
4.2	Khi nào nên Fine-tune?	12
4.3	Ba chiến lược Fine-tuning	13
4.4	Discriminative Learning Rates — Toán học & Trực giác	14
4.5	Quy trình Fine-tuning an toàn: Gradual Unfreezing	14
4.6	Giới thiệu Lab 2	18
5	Theo dõi Thực nghiệm, Ablation Study & Error Analysis	19
5.1	Câu hỏi mở đầu	19
5.2	Vì sao bắt buộc phải theo dõi thực nghiệm	19
5.3	Công cụ theo dõi thực nghiệm	19
5.4	Ablation Study — Vì sao cần và làm thế nào	21
5.5	Error Analysis — Nhìn vào nơi mô hình sai	22
6	Kỹ thuật Hiện đại — PEFT, LoRA & QLoRA	23
6.1	Câu hỏi mở đầu	23
6.2	Vấn đề: Full Fine-tuning không còn khả thi	23
6.3	Ý tưởng nền tảng: cập nhật trọng số có “hạng thấp”	23
6.4	LoRA — Low-Rank Adaptation	24
6.4.1	Weight Merging — vì sao LoRA “miễn phí” khi suy luận	25
6.5	QLoRA — Quantized LoRA	28
6.6	Họ kỹ thuật PEFT — bức tranh toàn cảnh	33
6.6.1	Hai trường phái: sửa trọng số vs thêm đầu vào ảo	34
6.7	So sánh: Full Fine-tuning vs LoRA	35
6.8	“Fine-tuning” trong kỷ nguyên LLM: ba cấp độ học	35
6.9	Code mẫu & Giới thiệu Lab 3	36
7	Ứng dụng & Các Tình huống Thực tế	37
7.1	Câu hỏi mở đầu	37
7.2	Computer Vision	38
7.3	Xử lý Ngôn ngữ Tự nhiên (NLP)	38
7.4	Đa phương thức (Multimodal)	38
7.5	Gợi ý Guest Lecture	39
8	Đạo đức & Thiên kiến trong Mô hình Pre-trained	39
9	Tổng kết & Dự án Cuối kỳ	40
9.1	Sơ đồ ra quyết định	40
9.2	Những điều cốt lõi cần nhớ	40
9.3	Dự án cuối kỳ (Create)	41

1. Mục tiêu & Bản đồ học tập

1.1 Mục tiêu học tập (Learning Objectives)

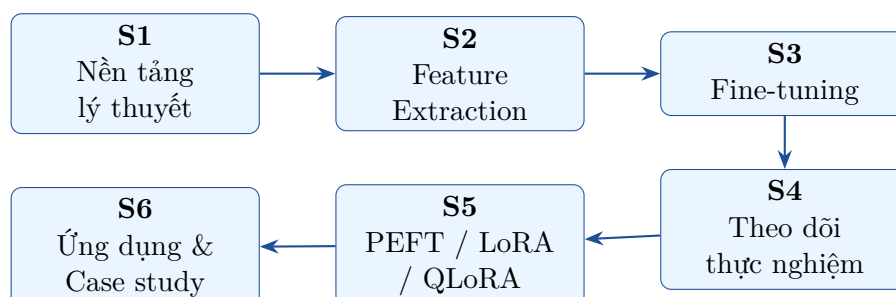
Sau khi hoàn thành module này, sinh viên có khả năng:

- **(Understand)** Giải thích chính xác ba khái niệm trung tâm: *Transfer Learning*, *Feature Extraction*, và *Fine-tuning*; cùng quan hệ phân cấp giữa chúng.
- **(Understand)** Mô tả được hiện tượng *domain shift*, *task shift*, và lý do vì sao biểu diễn (representation) học từ dữ liệu lớn có thể tái sử dụng.
- **(Apply)** Thiết kế và triển khai pipeline Transfer Learning hoàn chỉnh trên cả Computer Vision (ResNet/EfficientNet) và NLP (BERT).
- **(Apply)** Áp dụng các kỹ thuật tham số hiệu quả hiện đại: PEFT, LoRA, QLoRA, Adapter.
- **(Analyze)** Phân tích trade-off giữa hiệu năng (performance), chi phí tính toán (computational cost), và hiệu quả dữ liệu (data efficiency); đọc và phê bình một ablation study.
- **(Create)** Thiết kế giải pháp Transfer Learning cho một bài toán domain-specific thực tế và đề xuất cải tiến mô hình.

1.2 Kiến thức tiên quyết (Prerequisites)

- Lập trình Python và một framework deep learning (PyTorch khuyến nghị; TensorFlow chấp nhận được).
- CNN cơ bản: kiến trúc ResNet, EfficientNet; khái niệm receptive field.
- Kiến trúc Transformer và mô hình BERT.
- Tối ưu hóa: Gradient Descent, hiện tượng Overfitting, các kỹ thuật Regularization.
- Quy trình thực nghiệm: chia tập dữ liệu (train/val/test), các metric đánh giá.

1.3 Bản đồ module



Luồng học: từ vì sao ($S1$) $\rightarrow$ hai chiến lược cốt lõi ($S2$ – $S3$) $\rightarrow$ kỷ luật thực nghiệm ($S4$) $\rightarrow$ kỹ thuật tiên tiến ($S5$) $\rightarrow$ áp dụng thực tế ($S6$).

2. Nền tảng Lý thuyết của Transfer Learning

2.1 Warm-up: Một câu hỏi tưởng dễ

Câu hỏi gợi mở

Giả sử bạn cần xây một bộ phân loại 5 loài hoa lan quý hiếm, nhưng bạn chỉ có **5000 ảnh** (1000 ảnh mỗi loài). Bạn quyết định huấn luyện một ResNet-50 (khoảng 25 triệu tham số) *từ đầu* (from scratch).

Hãy dự đoán: Điều gì sẽ xảy ra với accuracy trên tập validation? Vì sao? Trước khi đọc tiếp, hãy viết ra giấy ít nhất hai lý do.

Hầu hết sinh viên sẽ trả lời “mô hình bị overfitting”. Đúng, nhưng đó mới là *triệu chứng*. Ta cần truy ngược về *nguyên nhân gốc*. Hãy bắt đầu bằng một *trực giác đếm tham số* (lưu ý: đây là cách nói trực giác, không phải lập luận chặt chẽ — xem cảnh báo ngay sau):

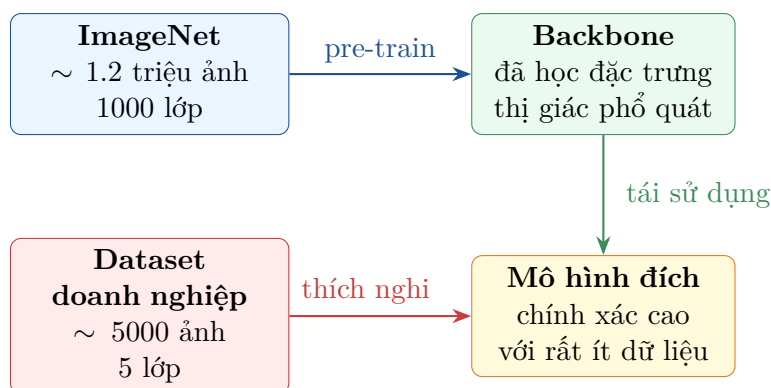
- ResNet-50 có $\sim 25 \times 10^6$ tham số tự do cần ước lượng.
- Tập dữ liệu có 5000 ảnh. Mỗi ảnh $224 \times 224 \times 3 \approx 1.5 \times 10^5$ số, nhưng *thông tin giám sát* (supervisory signal) thực sự chỉ là *nhãn* — khoảng $\log_2(5) \approx 2.3$ bit thông tin mỗi ảnh.
- Tổng tín hiệu giám sát: $\sim 5000 \times 2.3 \approx 1.2 \times 10^4$ bit. Con số này *nhỏ hơn nhiều* so với số tham số 25×10^6 — mô hình có thừa “dung lượng” để ghi nhớ thuộc lòng tập train.

Insight cần đạt

Huấn luyện từ đầu khó **không phải** vì thuật toán tối ưu yếu, mà vì **lượng thông tin giám sát quá ít để định hướng cho một không gian giả thuyết quá biểu cảm**. Với 5000 ảnh, mạng có vô số cách khớp hoàn hảo tập train; inductive bias của CNN giúp loại bớt các lời giải “xấu”, nhưng vẫn không đủ. Transfer Learning ra đời để bổ sung phần thông tin còn thiếu đó — bằng tri thức học từ dữ liệu lớn.

2.2 Động lực: ImageNet và “đứng trên vai người khổng lồ”

Bộ dữ liệu ImageNet (phiên bản ILSVRC) chứa khoảng **1.2 triệu ảnh** thuộc 1000 lớp. Một mô hình huấn luyện trên đó đã “nhìn” đủ nhiều cạnh, góc, kết cấu (texture), hình dạng, và bộ phận vật thể để học được một *bộ trích xuất đặc trưng thị giác phổ quát*.



Phép loại suy (Analogy)

Transfer Learning giống một học sinh giỏi Toán chuyển sang học Vật lý: bạn không bắt em ấy học lại từ phép cộng. Tư duy logic, kỹ năng giải phương trình, khả năng trừu tượng hóa — tất cả *chuyển giao* sang môn mới. Em ấy chỉ cần học *phần đặc thù* của Vật lý (định luật, đơn vị, thí nghiệm). Pre-trained backbone chính là “nền tảng tư duy” đó.

2.3 Định nghĩa hình thức

Khái niệm cốt lõi

Miền (Domain) $\mathcal{D} = \{\mathcal{X}, P(X)\}$ gồm không gian đặc trưng $\mathcal{X}$ và phân phối biên $P(X)$.

Tác vụ (Task) $\mathcal{T} = \{\mathcal{Y}, f(\cdot)\}$ gồm không gian nhãn $\mathcal{Y}$ và hàm dự đoán f (tương đương phân phối có điều kiện $P(Y | X)$).

Transfer Learning: Cho miền nguồn $\mathcal{D}_S$ với tác vụ $\mathcal{T}_S$, và miền đích $\mathcal{D}_T$ với tác vụ $\mathcal{T}_T$. Transfer Learning nhằm cải thiện việc học hàm dự đoán f_T trên miền đích bằng cách *khai thác tri thức* từ $(\mathcal{D}_S, \mathcal{T}_S)$, trong điều kiện $\mathcal{D}_S \neq \mathcal{D}_T$ hoặc $\mathcal{T}_S \neq \mathcal{T}_T$.

Từ định nghĩa này ta thấy có hai kiểu “lệch” (shift) mà sinh viên cần phân biệt rõ:

- **Domain shift** ($\mathcal{D}_S \neq \mathcal{D}_T$): cùng tác vụ, khác phân phối dữ liệu. Ví dụ: cùng là “phân loại chó/mèo” nhưng nguồn là ảnh chụp studio, đích là ảnh điện thoại trong điều kiện thiếu sáng.
- **Task shift** ($\mathcal{T}_S \neq \mathcal{T}_T$): cùng dữ liệu hoặc dữ liệu tương tự, khác mục tiêu. Ví dụ: nguồn phân loại 1000 lớp ImageNet, đích là 5 loài hoa lan — không gian nhãn hoàn toàn khác.

Câu hỏi gợi mở

Một mô hình pre-train trên ảnh ImageNet (ảnh tự nhiên ban ngày) được đem dùng cho ảnh X-quang ngược y khoa. Đây là domain shift, task shift, hay cả hai? Mức độ “lệch” này gợi ý gì cho chiến lược ta nên chọn?

2.4 Phân biệt ba khái niệm trung tâm

Đây là bảng mà sinh viên cần thuộc ý nghĩa.

Khái niệm	Ý nghĩa	Cập nhật trọng số backbone	Dữ liệu cần
Transfer Learning	Thuật ngữ ô (umbrella) — mọi cách mượn tri thức từ mô hình khác	Có / Không	Ít
Feature Extraction	Dùng backbone như một bộ trích xuất đặc trưng <i>cố định</i>	Không (freeze)	Rất ít
Fine-tuning	Huấn luyện tiếp backbone để thích nghi miền đích	Có (một phần / toàn bộ)	Trung bình

Lưu ý quan trọng

Lỗi khái niệm phổ biến của sinh viên: coi “Transfer Learning” và “Fine-tuning” là đồng nghĩa. **Không phải vậy.** Transfer Learning là khái niệm *rộng*; Feature Extraction và Fine-tuning là hai *chiến lược cụ thể* nằm bên trong nó. Một câu trả lời chính xác trong bài thi luôn bắt đầu bằng việc nói rõ quan hệ phân cấp này.

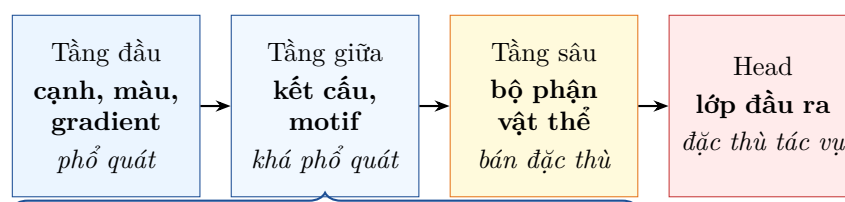
2.5 Toán học đằng sau: vì sao biểu diễn lại tái sử dụng được

Trực giác cốt lõi: một mạng sâu là một *hàm hợp* (composition) phân tách dần dữ liệu thành các biểu diễn trừu tượng hơn. Viết mạng dưới dạng:

$$f(x) = h \circ g_L \circ g_{L-1} \circ \cdots \circ g_1(x),$$

trong đó $g_1, \dots, g_L$ là các tầng backbone và h là đầu phân loại (head). Đặt $\phi(x) = g_L \circ \cdots \circ g_1(x)$ là *biểu diễn* (representation) học được.

Giả thuyết nền tảng của Transfer Learning là: **ánh xạ biểu diễn ϕ phụ thuộc chủ yếu vào cấu trúc thống kê của dữ liệu tự nhiên, không phải vào tác vụ cụ thể.** Các tầng đầu học những bộ lọc gần như phổ quát (cạnh, màu, gradient — tương tự bộ lọc Gabor); các tầng giữa học bộ phận (mắt, bánh xe, cánh hoa); chỉ các tầng cuối và head mới thực sự *task-specific*.



Backbone — tái sử dụng tốt

Có thể nhìn Transfer Learning qua lăng kính *regularization*. Khi huấn luyện từ đầu, ta tìm θ trong toàn không gian tham số Θ . Khi khởi tạo từ trọng số pre-trained θ_0 và dùng learning rate nhỏ kèm early stopping, gradient descent *xuất phát từ* θ_0 và chỉ kịp đi một quãng ngắn — nên nghiệm cuối cùng nằm trong một *lân cận* của θ_0 . Về mặt trực giác, điều này *tương tự* việc thêm một số hạng phạt khoảng cách:

$$\min_{\theta} \mathcal{L}_{\text{task}}(\theta) \quad \text{hành xử na ná như} \quad \min_{\theta} \mathcal{L}_{\text{task}}(\theta) + \lambda \|\theta - \theta_0\|^2.$$

Pre-trained weights đóng vai trò một *prior* mạnh: ta tin rằng nghiệm tốt nằm gần θ_0 .

Lưu ý quan trọng

Đây là tương đồng trực giác, không phải đồng nhất toán học. Cần phân biệt ba khái niệm để tránh sai lầm:

- **Implicit regularization.** LR nhỏ + early stopping tạo ra một dạng regularization *ngầm* dọc theo quỹ đạo tối ưu — nó giữ nghiệm gần θ_0 nhưng *không* tương ứng với bất kỳ số hạng phạt tường minh nào trong hàm mất mát.
- **Weight decay chuẩn.** Nếu bạn bật weight decay thông thường (như mặc định trong Adam/SGD), số hạng phạt là $\lambda \|\theta\|^2$ — nó kéo trọng số về **gốc tọa độ** 0, *không* về θ_0 .
- **L2-SP (L2 Starting Point).** Muốn có *đúng* số hạng $\lambda \|\theta - \theta_0\|^2$ như công thức trên, ta phải *chủ động sửa* thuật toán tối ưu để phạt khoảng cách tới θ_0 thay vì tới 0. Kỹ thuật này có tên riêng là L2-SP và là một lựa chọn thiết kế *tường minh*.

Tóm lại: công thức trên là một *mô hình tinh thần* hữu ích, nhưng nó chỉ trở thành *sự thật toán học chính xác* khi ta dùng L2-SP.

Insight cần đạt

Transfer Learning hoạt động được vì hai điều: (1) **tính phân cấp** của biểu diễn — đặc trưng cấp thấp có tính phổ quát; và (2) **vai trò prior** của trọng số pre-trained — nó thu hẹp không gian tìm kiếm về vùng “đã biết là tốt”, biến một bài toán nghèo thông tin giám sát thành một bài toán giải được với ít dữ liệu.

2.6 Khi miền lệch nhưng tác vụ không đổi: Domain Adaptation

Một quan sát quan trọng: ba kỹ thuật trọng tâm của module này — Feature Extraction, Fine-tuning, và sau này LoRA — đều xử lý rất tốt *task shift* (đổi tác vụ, ví dụ 1000 lớp ImageNet $\rightarrow$ 5 loài hoa lan). Nhưng còn trường hợp ngược lại: *tác vụ giữ nguyên, chỉ miền dữ liệu thay đổi?*

Ví dụ minh họa

Một mô hình phát hiện người đi bộ được huấn luyện trên ảnh *ban ngày*, nay cần hoạt động trên ảnh *ban đêm*. Tác vụ y hệt (vẫn là “phát hiện người đi bộ”), nhưng phân phối đầu vào $P(X)$ đổi hoàn toàn. Vấn đề nan giải: ta thường có *rất nhiều* ảnh

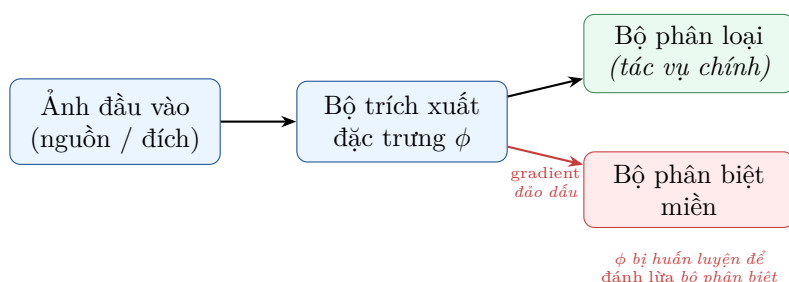
ban đêm **không** **nhân**, nhưng *gần như không có* ảnh ban đêm *có* **nhân**. Fine-tuning kiểu thông thường cần nhân ở miền đích — nên ở đây nó bẻ tắc.

Khái niệm cốt lõi

Domain Adaptation (DA) là nhánh chuyên giải bài toán *domain shift* khi tác vụ không đổi, đặc biệt trong kịch bản *unsupervised* — miền nguồn có nhãn, miền đích chỉ có dữ liệu thô. Ý tưởng cốt lõi: ép mô hình học một biểu diễn $\phi(x)$ sao cho phân phối đặc trưng của hai miền *trùng khớp* nhau — lúc đó bộ phân loại học từ miền nguồn sẽ tự động dùng được cho miền đích.

Hai họ phương pháp kinh điển sinh viên nên biết:

- **Khớp phân phối tường minh** — **MMD (Maximum Mean Discrepancy)**. Ta đo “khoảng cách” giữa phân phối đặc trưng nguồn và đích bằng một đại lượng thống kê gọi là MMD, rồi *thêm nó vào hàm mất mát* như một số hạng phạt. Mô hình vừa học phân loại tốt trên nguồn, vừa bị buộc kéo hai phân phối đặc trưng lại gần nhau.
- **Khớp phân phối kiểu đối kháng** — **DANN (Domain-Adversarial Neural Networks)**. Thêm một “bộ phân biệt miền” (domain discriminator) cố đoán mỗi đặc trưng đến từ miền nào. Bộ trích xuất đặc trưng thì bị huấn luyện để *đánh lừa* bộ phân biệt đó. Kết quả của trò chơi đối kháng này: đặc trưng trở nên *bất khả phân biệt theo miền* — tức hai miền đã được căn chỉnh.



Insight cần đạt

Domain Adaptation và Fine-tuning giải hai bài toán *khác nhau*: Fine-tuning thích nghi sang *tác vụ mới* (cần nhân ở miền đích); Domain Adaptation thích nghi sang *miền mới cùng tác vụ* (có thể *không* cần nhân ở miền đích). Trong thực tế, khi gặp một bài toán mới, câu hỏi đầu tiên luôn là: “cái gì đã đổi — tác vụ, miền, hay cả hai?” — câu trả lời quyết định ta chọn nhánh kỹ thuật nào.

Ghi chú giảng viên

Module này tập trung vào Fine-tuning và PEFT (xử lý task shift) vì đó là kỹ thuật phổ biến nhất trong thực hành hiện nay. Domain Adaptation được giới thiệu ở mức *nhận thức* — sinh viên cần biết nó *tồn tại* và biết *khi nào* cần tới nó. Một module chuyên sâu về DANN, MMD, CORAL có thể là hướng mở rộng sau này.

Bài tập

$(\mathcal{D}_T, \mathcal{T}_T)$ trong đó *chỉ có* domain shift, và một cặp khác *chỉ có* task shift. Giải thích.

Bài tập 1.2 (Analyze). Vì sao Transfer Learning từ ImageNet thường *ít hiệu quả hơn* khi áp dụng cho ảnh y khoa so với khi áp dụng cho ảnh sản phẩm thương mại điện tử? Liên hệ với khái niệm khoảng cách giữa các miền.

Bài tập 1.3 (Create). Đề xuất một thí nghiệm để *kiểm chứng* giả thuyết “tầng đầu học đặc trưng phổ quát”. Bạn sẽ đo cái gì, và kỳ vọng kết quả ra sao?

3. Feature Extraction — Đào sâu & Thực hành

3.1 Câu hỏi mở đầu

Câu hỏi gợi mở

Nếu chúng ta “đóng băng” (freeze) toàn bộ backbone của ResNet-50 và chỉ huấn luyện một lớp tuyến tính cuối cùng, thì về mặt toán học, ta thực chất đang giải bài toán học máy gì? Và bài toán đó có còn là “deep learning” nữa không?

3.2 Vì sao tầng đầu học low-level, tầng sau học high-level?

Đây không phải một quy ước do con người áp đặt, mà là *hệ quả tự nhiên* của ba yếu tố:

1. **Receptive field tăng dần.** Một neuron ở tầng đầu chỉ “nhìn” được một vùng nhỏ (ví dụ 3×3 pixel) — nó vật lý không thể biểu diễn khái niệm “con mèo” vì khái niệm đó cần thông tin trên toàn ảnh. Tầng càng sâu, receptive field càng lớn, mới đủ ngữ cảnh để biểu diễn vật thể hoàn chỉnh.
2. **Tính hợp thành (compositionality).** Mạng buộc phải xây khái niệm phức tạp từ khái niệm đơn giản: cạnh $\rightarrow$ góc $\rightarrow$ đường viền $\rightarrow$ bộ phận $\rightarrow$ vật thể. Không có đường tắt.
3. **Áp lực tối ưu.** Gradient descent ưu tiên các đặc trưng *dùng lại được nhiều lần*. Cạnh và kết cấu hữu ích cho hầu hết mọi lớp, nên được học sớm và ổn định; đặc trưng đặc thù lớp chỉ hữu ích cục bộ nên xuất hiện muộn.

Ứng dụng thực tế

Trong thực nghiệm kinh điển của Yosinski và cộng sự (2014), người ta đo độ “chuyển giao được” (transferability) của từng tầng. Kết quả: các tầng đầu chuyển giao gần như hoàn hảo sang tác vụ mới; độ chuyển giao *giảm dần* theo độ sâu. Đây là bằng chứng thực nghiệm trực tiếp cho phân cấp đặc trưng đã nói ở Section 1.

3.3 Cơ chế Freezing: `requires_grad = False`

Trong PyTorch, mỗi tham số (Parameter) có thuộc tính `requires_grad`. Khi đặt nó bằng `False`, ba điều xảy ra:

- Autograd *không* tính gradient cho tham số đó (tiết kiệm bộ nhớ và thời gian).
- Optimizer *không* cập nhật nó (dù nó có nằm trong danh sách tham số).
- Tham số được giữ *nguyên giá trị pre-trained* suốt quá trình huấn luyện.

Lưu ý quan trọng

Sai lầm kỹ thuật rất thường gặp: đặt `requires_grad = False` cho backbone *nhưng* vẫn truyền `model.parameters()` vào optimizer. Điều này không gây lỗi, nhưng lãng phí — và nguy hiểm hơn: nếu sau đó bạn *quên* và mở băng (unfreeze) lại, optimizer sẽ đột ngột cập nhật backbone với learning rate dành cho head (thường quá lớn), phá hủy trọng số pre-trained. **Quy tắc:** chỉ đưa vào optimizer những tham số có `requires_grad = True`.

3.4 Code mẫu: Feature Extraction với ResNet-50

Listing 1: Thiết lập Feature Extraction trong PyTorch

```

1 import torch
2 import torch.nn as nn
3 from torchvision import models
4
5 # 1. Tải backbone đã pre-train trên ImageNet
6 model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V2
7 )
8
9 # 2. Đóng băng toàn bộ backbone
10 for param in model.parameters():
11     param.requires_grad = False
12
13 # 3. Thay thế lớp phân loại cuối (head) bằng lớp mới cho 5 lớp hoa
14     lan
15 #     Lớp mới mặc định có requires_grad = True
16 num_features = model.fc.in_features          # 2048 đối với ResNet-50
17 model.fc = nn.Linear(num_features, 5)        # 5 lớp hoa lan
18
19 # 4. Chỉ đưa THAM SỐ CÓ requires_grad=True vào optimizer
20 trainable_params = [p for p in model.parameters() if p.requires_grad]
21 optimizer = torch.optim.Adam(trainable_params, lr=1e-3)
22
23 # Kiểm tra: chỉ ~10k tham số được huấn luyện thay vì 25 triệu
24 n_trainable = sum(p.numel() for p in trainable_params)
25 n_total     = sum(p.numel() for p in model.parameters())
26 print(f"Huấn luyện {n_trainable:,} / {n_total:,} tham số "
27       f"({100*n_trainable/n_total:.3f}%)")

```

Ghi chú giảng viên

Nên cho sinh viên chạy đúng dòng `print` cuối cùng và *quan sát con số*: chỉ khoảng 0.04% tham số được huấn luyện. Đây là khoảnh khắc “aha” — nó cho thấy vì sao Feature Extraction nhanh, nhẹ, và cực kỳ chống overfitting.

3.5 Giới thiệu Lab 1**Bài tập****Lab 1 — So sánh: Train from scratch vs Feature Extraction.**

Dữ liệu: Flowers-102 hoặc Food-101 (chọn một; cả hai có sẵn trong `torchvision.datasets`).

Yêu cầu thực hiện:

1. Nhánh A — Huấn luyện ResNet-50 *từ đầu* (khởi tạo ngẫu nhiên).
2. Nhánh B — Feature Extraction: backbone đóng băng, chỉ train head.
3. Với cả hai nhánh, ghi lại: accuracy trên validation, thời gian mỗi epoch, dung lượng bộ nhớ GPU đỉnh (`torch.cuda.max_memory_allocated`).
4. Vẽ trên cùng một biểu đồ: đường cong accuracy theo epoch của A và B.

Câu hỏi phân tích bắt buộc:

- Nhánh nào đạt accuracy cao hơn *trong 5 epoch đầu*? Vì sao?
- Khoảng cách giữa train accuracy và val accuracy của nhánh nào lớn hơn? Điều này nói lên gì về overfitting?
- Nếu dataset chỉ còn 1/10 kích thước, bạn dự đoán khoảng cách hiệu năng giữa A và B sẽ *nới rộng* hay *thu hẹp*? Giải thích trước, rồi kiểm chứng.

3.6 Visualization: nhìn vào không gian đặc trưng

Một mô hình tốt không chỉ cho accuracy cao — nó còn tạo ra một *không gian đặc trưng có cấu trúc*. Hai công cụ trực quan hóa nên dạy cho sinh viên:

- **t-SNE / UMAP.** Lấy vector đặc trưng $\phi(x)$ (đầu ra backbone, trước head) cho mọi ảnh validation, chiếu xuống 2D. Nếu Feature Extraction hiệu quả, các điểm cùng lớp sẽ *tự thành cụm tách biệt* — ngay cả khi head chưa được huấn luyện. Đây là bằng chứng trực quan rằng backbone đã “hiểu” dữ liệu.
- **Grad-CAM.** Trực quan hóa *vùng ảnh* mà mô hình dựa vào để ra quyết định. Dùng để phát hiện lỗi tinh vi: ví dụ mô hình phân loại “hoa lan” nhưng thực chất nhìn vào *cái chậu* hoặc *nền* — một dạng “đường tắt” (shortcut learning) nguy hiểm.

Listing 2: Trích xuất đặc trưng cho t-SNE

```
1 import numpy as np
```

```

2 from sklearn.manifold import TSNE
3
4 # Tao backbone-only (bo head) de lay vector dac trung
5 feature_extractor = nn.Sequential(*list(model.children())[:-1])
6 feature_extractor.eval()
7
8 feats, labels = [], []
9 with torch.no_grad():
10     for images, ys in val_loader:
11         z = feature_extractor(images)          # (B, 2048, 1, 1)
12         feats.append(z.flatten(1).cpu().numpy())
13         labels.append(ys.numpy())
14
15 feats = np.concatenate(feats)                # (N, 2048)
16 labels = np.concatenate(labels)
17 embedding = TSNE(n_components=2, perplexity=30).fit_transform(feats)
18 # embedding gio co the ve scatter plot, to mau theo labels

```

Câu hỏi gợi mở

Quay lại câu hỏi đầu section: khi freeze backbone và chỉ train một lớp Linear, ta đang giải bài toán gì? — Đó là **hồi quy logistic đa lớp** trên đặc trưng $\phi(x)$ cố định. Vậy: nếu các cụm trong biểu đồ t-SNE đã tách biệt đẹp, tại sao ta vẫn cần huấn luyện lớp Linear đó? Ranh giới quyết định nào nó đang đi tìm?

Insight cần đạt

Feature Extraction tách bài toán làm hai phần *độc lập*: backbone (đã cố định) lo việc *biểu diễn*; head (được huấn luyện) lo việc *phân tách*. Vì head rất nhỏ, nó cần ít dữ liệu và gần như không thể overfit. Đánh đổi: nếu miền đích quá khác miền nguồn, backbone cố định sẽ tạo ra biểu diễn *không phù hợp* — và lúc đó không head nào cứu được. Đó chính là lúc ta cần Fine-tuning.

4. Fine-tuning & Các Thực hành Tốt nhất

4.1 Câu hỏi mở đầu

Câu hỏi gợi mở

Feature Extraction đã nhanh, nhẹ, chống overfitting tốt. Vậy *tại sao* ta còn cần Fine-tuning? Trong tình huống nào việc “động vào” backbone — vốn rủi ro — lại đáng để đánh đổi?

4.2 Khi nào nên Fine-tune?

Quyết định không nên dựa vào cảm tính. Hãy dùng *ma trận quyết định* dựa trên hai trục: **lượng dữ liệu đích** và **độ tương đồng giữa miền nguồn–đích**.

	Miền đích GIỐNG nguồn	Miền đích KHÁC nguồn
Dữ liệu ÍT	Feature Extraction. Backbone đã phù hợp; fine-tune chỉ làm tăng nguy cơ overfitting.	Khó nhất. Feature Extraction từ vài tầng <i>giữa</i> ; cân nhắc thu thập thêm dữ liệu hoặc tăng cường dữ liệu (augmentation) mạnh.
Dữ liệu NHIỀU	Fine-tune vài tầng cuối. Đủ dữ liệu để tinh chỉnh an toàn.	Fine-tune toàn bộ (full fine-tuning) với learning rate phân biệt. Có thể cân nhắc cả train from scratch.

Câu hỏi gợi mở

Trả lời câu hỏi treo từ Section 1: mô hình ImageNet đem dùng cho ảnh X-quang ngược — đó là *cả* domain shift *lẫn* task shift, và mức độ lệch rất lớn (ảnh y khoa là ảnh xám, kết cấu hoàn toàn khác ảnh tự nhiên). Theo ma trận trên: nếu có nhiều dữ liệu X-quang, hãy full fine-tuning; nếu ít, hãy trích đặc trưng từ tầng giữa và augmentation mạnh. Feature Extraction “ngây thơ” từ tầng cuối sẽ hoạt động kém.

4.3 Ba chiến lược Fine-tuning

Khái niệm cốt lõi

Chiến lược 1 — Chỉ train classifier (head). Tương đương Feature Extraction. Rẻ nhất, an toàn nhất, dùng làm *baseline bắt buộc*.

Chiến lược 2 — Fine-tune vài tầng cuối. Đóng băng tầng đầu (đặc trưng phổ quát), mở băng các tầng sâu (đặc trưng bán đặc thù) và head. Cân bằng tốt giữa khả năng thích nghi và rủi ro overfitting.

Chiến lược 3 — Fine-tune toàn bộ với learning rate phân biệt. Mở băng toàn mạng nhưng dùng learning rate *khác nhau* cho từng nhóm tầng: tầng đầu LR rất nhỏ (đụng nhẹ), head LR lớn hơn (học mạnh).

Phép loại suy (Analogy)

Ba chiến lược tương ứng ba mức độ “học thêm” của học sinh giỏi Toán chuyển sang Vật lý: **(1)** chỉ học công thức Vật lý, giữ nguyên mọi tư duy cũ; **(2)** điều chỉnh một số thói quen suy luận gần với Vật lý; **(3)** tinh chỉnh *cả* tư duy nền tảng — nhưng cực kỳ thận trọng, vì tư duy nền tốt rồi, đụng mạnh là hỏng.

4.4 Discriminative Learning Rates — Toán học & Trực giác

Ý tưởng cốt lõi: tầng càng gần đầu vào càng *phổ quát* và càng *không cần* thay đổi, nên nhận learning rate nhỏ; tầng càng gần đầu ra càng cần thích nghi, nhận learning rate lớn hơn. Một công thức phổ biến (dùng trong fast.ai) là cho LR *giảm theo cấp số nhân* từ head về tầng đầu:

$$\eta_\ell = \eta_{\text{base}} \cdot \gamma^{(L-\ell)}, \quad 0 < \gamma < 1,$$

với ℓ là chỉ số tầng ($\ell = L$ là head), γ thường khoảng 0.3–0.5.

Lưu ý quan trọng

Vì sao backbone cần LR rất nhỏ (10^{-4} đến 10^{-5})? Trọng số pre-trained đang nằm ở một *cực tiểu tốt* trong không gian mất mát. Một bước gradient lớn sẽ “hất văng” trọng số ra khỏi vùng tốt đó — hiện tượng gọi là **catastrophic forgetting** (quên thảm họa): mạng quên sạch tri thức ImageNet trước khi kịp học tri thức mới. LR nhỏ giữ ta *trong lân cận* của nghiệm pre-trained — chính là ý tưởng “prior” ở Section 1.

Listing 3: Discriminative learning rates với parameter groups

```

1 import torch
2
3 # Gia su model gom 3 khoi: backbone_early, backbone_late, head
4 optimizer = torch.optim.AdamW([
5     {"params": model.backbone_early.parameters(), "lr": 1e-5}, #
6     {"params": model.backbone_late.parameters(), "lr": 1e-4}, #
7     {"params": model.head.parameters(), "lr": 1e-3}, #
8 ], weight_decay=1e-2)
9
10 # Scheduler: warm-up roi giam dan (cosine) -- rat quan trong khi fine
11 # tune
12 from torch.optim.lr_scheduler import CosineAnnealingLR
13 scheduler = CosineAnnealingLR(optimizer, T_max=num_epochs)

```

4.5 Quy trình Fine-tuning an toàn: Gradual Unfreezing

Không nên mở băng toàn mạng ngay từ epoch 0. Quy trình *mở băng dần* (gradual unfreezing) ổn định hơn nhiều:

1. **Giai đoạn 1.** Đóng băng toàn bộ backbone, chỉ train head vài epoch cho tới khi head “ấm” lên. Lý do: head khởi tạo ngẫu nhiên tạo gradient rất lớn và nhiễu; nếu mở băng backbone ngay, gradient nhiễu này sẽ truyền ngược và phá backbone.
2. **Giai đoạn 2.** Mở băng nhóm tầng sâu nhất, train tiếp với LR nhỏ.
3. **Giai đoạn 3.** Mở băng dần các nhóm còn lại, dùng discriminative LR cho toàn mạng.

```

1      """
2  Minh hoa quy trình Fine-tuning an toan: Gradual Unfreezing
3  3 giai doan: (1) Warm-up head -> (2) Unfreeze tang sau -> (3)
4      Unfreeze toan mang + discriminative LR
5      """
6
7  import torch
8  import torch.nn as nn
9  from torchvision import models
10
11  # =====
12  # 1. CHUAN BI MODEL: backbone (pretrained) + head (random init)
13  # =====
14  def build_model(num_classes=10):
15      model = models.resnet18(weights=models.ResNet18_Weights.
16          IMAGENET1K_V1)
17      # Thay head bang lop moi -> head nay khoi tao NGAU NHIEN
18      in_features = model.fc.in_features
19      model.fc = nn.Linear(in_features, num_classes)
20      return model
21
22  # =====
23  # 2. HAM TIEN ICH: dong/mo bang theo nhom tang
24  # =====
25  def set_requires_grad(module, trainable: bool):
26      """Bat/tat gradient cho mot nhom tang."""
27      for param in module.parameters():
28          param.requires_grad = trainable
29
30
31  def freeze_all_backbone(model):
32      """Dong bang toan bo backbone, chi chua head (fc)."""
33      for name, param in model.named_parameters():
34          param.requires_grad = name.startswith("fc")
35
36
37  def count_trainable(model):
38      trainable = sum(p.numel() for p in model.parameters() if p.
39          requires_grad)
40      total = sum(p.numel() for p in model.parameters())
41      return trainable, total
42
43  # ResNet18 co cac "nhom" tang theo do sau:
44  # conv1/bn1 (nong) -> layer1 -> layer2 -> layer3 -> layer4 (sau nhat)
45  #   -> fc (head)
46  # Quy uoc: nhom cang "sau" (gan head) thi mo bang cang som.
47

```

```

48 # =====
49 # 3. VONG LAP TRAIN GIA LAP (thay bang dataloader that của bạn)
50 # =====
51 def train_one_phase(model, optimizer, phase_name, num_epochs, device)
52 :
53     model.train()
54     criterion = nn.CrossEntropyLoss()
55     trainable, total = count_trainable(model)
56     print(f"\n=== {phase_name} ===")
57     print(f"    Tham số train được: {trainable:,} / {total:,} "
58           f"({100*trainable/total:.1f}%)")
59
60     for epoch in range(num_epochs):
61         # ---- Du lieu gia lap: thay bang for x, y in dataloader ----
62         x = torch.randn(16, 3, 224, 224, device=device)
63         y = torch.randint(0, 10, (16,), device=device)
64
65         optimizer.zero_grad()
66         out = model(x)
67         loss = criterion(out, y)
68         loss.backward()
69
70         # Gradient clipping: phong gradient nhieu lon (dac biet giai
71         # doan 1)
72         torch.nn.utils.clip_grad_norm_(
73             [p for p in model.parameters() if p.requires_grad],
74             max_norm=1.0
75         )
76         optimizer.step()
77         print(f"    epoch {epoch+1}/{num_epochs} - loss: {loss.item()
78               :.4f}")
79
80 # =====
81 # 4. QUY TRINH 3 GIAI DOAN
82 # =====
83 def gradual_unfreezing_finetune():
84     device = "cuda" if torch.cuda.is_available() else "cpu"
85     model = build_model(num_classes=10).to(device)
86
87     # -----
88     # GIAI DOAN 1: Dong bang toan bo backbone, chi "ham nong" head
89     # -----
90     # Ly do: head random tao gradient lon & nhieu. Neu mo bang
91     # backbone
92     # ngay, gradient nhieu nay lan nguoc va pha hong dac trung
93     # pretrained.
94     freeze_all_backbone(model)
95     optimizer = torch.optim.Adam(
96         filter(lambda p: p.requires_grad, model.parameters()),
97         lr=1e-3, # LR lon duoc phep vi chi train head

```



```

93 )
94 train_one_phase(model, optimizer, "GIAI DOAN 1: Warm-up head",
95                 num_epochs=3, device=device)
96
97 # -----
98 # GIAI DOAN 2: Mo bang nhom tang SAU NHAT (layer4), train LR nho
99 # -----
100 # Head da "am" -> gradient on dinh hon. Mo them layer4 (gan head
    nhat).
101 set_requires_grad(model.layer4, True)
102 optimizer = torch.optim.Adam(
103     [
104         {"params": model.fc.parameters(), "lr": 1e-4},
105         {"params": model.layer4.parameters(), "lr": 1e-5}, # LR
            nho hon
106     ]
107 )
108 train_one_phase(model, optimizer, "GIAI DOAN 2: Unfreeze layer4",
109                 num_epochs=3, device=device)
110
111 # -----
112 # GIAI DOAN 3: Mo bang dan cac nhom con lai + discriminative LR
113 # -----
114 # Discriminative LR: tang cang NONG (dac trung tong quat, it can
    doi)
115 # -> LR cang nho; tang cang SAU (dac trung chuyen biet) -> LR lon
    hon.
116 set_requires_grad(model.conv1, True)
117 set_requires_grad(model.bn1, True)
118 set_requires_grad(model.layer1, True)
119 set_requires_grad(model.layer2, True)
120 set_requires_grad(model.layer3, True)
121
122 optimizer = torch.optim.Adam([
123     # nhom nong nhat -> LR rat nho
124     {"params": list(model.conv1.parameters()) +
125                list(model.bn1.parameters()) +
126                list(model.layer1.parameters()), "lr": 1e-6},
127     {"params": model.layer2.parameters(), "lr": 5e-6},
128     {"params": model.layer3.parameters(), "lr": 1e-5},
129     {"params": model.layer4.parameters(), "lr": 5e-5},
130     # head -> LR lon nhat
131     {"params": model.fc.parameters(), "lr": 1e-4},
132 ])
133 train_one_phase(model, optimizer, "GIAI DOAN 3: Unfreeze toan
    mang + discriminative LR",
134                 num_epochs=3, device=device)
135
136 print("\nHoan tat fine-tuning theo quy trinh Gradual Unfreezing."
137 )
138 return model

```

Insight cần đạt

Trật tự là tất cả: **làm ấm head** → **mở băng dần từ sâu ra nông** → **LR nhỏ cho tầng nông**. Vi phạm trật tự này là nguyên nhân số một khiến fine-tuning cho kết quả *tệ hơn* cả Feature Extraction — một nghịch lý làm nản lòng nhiều sinh viên.

4.6 Giới thiệu Lab 2**Bài tập**

Lab 2 — Fine-tune BERT cho phân loại cảm xúc (sentiment analysis).

Dữ liệu: một bộ dữ liệu cảm xúc tiếng Việt (ví dụ bình luận sản phẩm / đánh giá phim), hoặc tập tiếng Anh nếu chưa có. Khuyến nghị dùng checkpoint PhoBERT cho tiếng Việt.

Yêu cầu thực hiện:

1. Cài đặt cả ba chiến lược fine-tuning ở mục 4.3.
2. Với mỗi chiến lược, ghi: accuracy / F1 trên test, thời gian huấn luyện, bộ nhớ GPU đỉnh.
3. Thử ít nhất ba giá trị learning rate cho backbone (10^{-3} , 10^{-4} , 10^{-5}) và quan sát: giá trị nào gây catastrophic forgetting?

Bảng tổng hợp cần điền:

Chiến lược	Accuracy	F1	Thời gian	GPU mem
1 – Chỉ head	...	...	...	...
2 – Vài tầng cuối	...	...	...	...
3 – Toàn bộ + discrim. LR	...	...	...	...

Câu hỏi phân tích: Chiến lược nào cho tỉ lệ “điểm hiệu năng trên một đơn vị thời gian tính toán” tốt nhất? Trong điều kiện tài nguyên hạn chế, bạn sẽ chọn chiến lược nào, và đó có phải chiến lược cho accuracy cao nhất không?

Insight cần đạt

Kết thúc Section 3, sinh viên cần nắm được: Fine-tuning *không phải lúc nào cũng tốt hơn* Feature Extraction. Nó tốt hơn *khi và chỉ khi* có đủ dữ liệu và/hoặc miền đích lệch xa — và chỉ khi được thực hiện đúng kỹ thuật (làm ấm head, mở băng dần, LR nhỏ cho tầng nông).

5. Theo dõi Thực nghiệm, Ablation Study & Error Analysis

5.1 Câu hỏi mở đầu

Câu hỏi gợi mở

Bạn chạy 40 thí nghiệm fine-tuning trong một tuần, thay đổi learning rate, số tầng mở băng, kiểu augmentation. Đến cuối tuần, đồng nghiệp hỏi: “Cấu hình nào tốt nhất, và *vì sao?*”. Nếu bạn chỉ ghi kết quả rải rác trong các ô notebook và tên file `model_final_v2_real_final.pt`, bạn có trả lời được không? Điều gì đã sai?

5.2 Vì sao bắt buộc phải theo dõi thực nghiệm

Theo dõi thực nghiệm (experiment tracking) không phải là “việc làm cho đẹp”. Nó là điều kiện cần để công việc deep learning trở thành *khoa học* thay vì *thử-sai mù quáng*. Bốn lý do cốt lõi:

1. **Khả năng tái lập (reproducibility).** Một kết quả không tái lập được thì không phải kết quả. Tracking ghi lại *toàn bộ* điều kiện: seed, phiên bản dữ liệu, siêu tham số, commit code — đủ để bất kỳ ai chạy lại cũng ra cùng số.
2. **So sánh công bằng.** Khi 40 lần chạy được ghi theo cùng một định dạng, ta có thể đặt chúng cạnh nhau và biết *chính xác* cái gì khác nhau giữa hai lần chạy — thay vì đoán mò.
3. **Quy kết nguyên nhân (attribution).** Nếu accuracy tăng 3%, là nhờ learning rate mới hay nhờ augmentation mới? Không tracking thì không thể tách biệt.
4. **Tránh lãng phí.** Tracking giúp phát hiện sớm các lần chạy hỏng (loss NaN, val accuracy đứng yên) để dừng kịp thời, tiết kiệm GPU-giờ.

Ví dụ minh họa

Tình huống cụ thể. Một nhóm sinh viên báo cáo: “fine-tuning đạt 92%, tốt hơn feature extraction 88%”. Giảng viên hỏi seed nào. Hóa ra hai lần chạy dùng *hai seed khác nhau* và *hai lần chia train/val khác nhau*. Khi chạy lại với cùng seed, chênh lệch chỉ còn 0.5% — nằm trong khoảng nhiễu. “Kết quả” ban đầu là một **ảo giác do thiếu kiểm soát**. Đây chính xác là thứ experiment tracking ngăn chặn.

5.3 Công cụ theo dõi thực nghiệm

Khái niệm cốt lõi

TensorBoard. Công cụ trực quan hóa đi kèm hệ sinh thái PyTorch/TensorFlow. Mạnh ở biểu đồ loss/metric theo thời gian thực, đồ thị tính toán, histogram trọng số. Chạy cục bộ, miễn phí, nhẹ — phù hợp cho cá nhân và lớp học.

Weights & Biases (W&B / wandb). Nền tảng trên đám mây. Mạnh ở: bảng so

sánh nhiều lần chạy, sweeps (tìm kiếm siêu tham số tự động), lưu artifact (dataset, checkpoint), cộng tác nhóm, báo cáo chia sẻ được. Phù hợp cho dự án nhóm và nghiên cứu nghiêm túc.

Các lựa chọn khác: MLflow (mạnh về quản lý vòng đời mô hình), Neptune, Comet.

Listing 4: Theo dõi một lần chạy fine-tuning với Weights & Biases

```

1 import wandb
2
3 # 1. Khởi tạo một "run" -- ghi lại TOÀN BỘ cấu hình
4 wandb.init(
5     project="transfer-learning-flowers",
6     config={
7         "strategy": "fine_tune_last_2_blocks",
8         "backbone": "resnet50",
9         "lr_head": 1e-3, "lr_backbone": 1e-5,
10        "batch_size": 32, "epochs": 20,
11        "augmentation": "randaugment", "seed": 42,
12    },
13 )
14
15 for epoch in range(num_epochs):
16     train_loss = train_one_epoch(...)
17     val_loss, val_acc = evaluate(...)
18     # 2. Ghi metric theo từng epoch
19     wandb.log({"epoch": epoch, "train_loss": train_loss,
20              "val_loss": val_loss, "val_acc": val_acc})
21
22 # 3. Lưu checkpoint như một artifact có phiên bản
23 wandb.save("best_model.pt")
24 wandb.finish()

```

Listing 5: Tương đương với TensorBoard

```

1 from torch.utils.tensorboard import SummaryWriter
2 writer = SummaryWriter(log_dir="runs/ft_last2_lr1e-5_seed42")
3
4 for epoch in range(num_epochs):
5     train_loss = train_one_epoch(...)
6     val_loss, val_acc = evaluate(...)
7     writer.add_scalar("Loss/train", train_loss, epoch)
8     writer.add_scalar("Loss/val", val_loss, epoch)
9     writer.add_scalar("Acc/val", val_acc, epoch)
10 writer.close()
11 # Xem kết quả: tensorboard --logdir runs

```

Lưu ý quan trọng

Quy tắc đặt tên: tên run/log-dir phải *tự mô tả* — nhúng các siêu tham số chính vào tên (ft_last2_lr1e-5_seed42). Tuyệt đối tránh final, final2, real_final. Một cái tên tốt giúp bạn tìm lại đúng thí nghiệm sau ba tháng.

5.4 Ablation Study — Vì sao cần và làm thế nào**Khái niệm cốt lõi**

Ablation study là quy trình *tắt / bỏ đi từng thành phần một* của hệ thống để đo xem mỗi thành phần *thực sự* đóng góp bao nhiêu vào kết quả cuối. Nó trả lời câu hỏi: “*Cái gì làm cho mô hình hoạt động?*”. Có công cụ nào để theo dõi không?

Vì sao bắt buộc: một pipeline fine-tuning hiện đại có hàng chục thành phần — augmentation, discriminative LR, gradual unfreezing, scheduler, label smoothing, ...Nếu chỉ báo cáo “cả gói đạt 93%”, ta *không biết* thành phần nào quan trọng. Tệ hơn, có thể một thành phần đang *gây hại* nhưng bị các thành phần khác che lấp.

Ví dụ minh họa

Ví dụ ablation cho pipeline fine-tuning. Bắt đầu từ mô hình đầy đủ, lần lượt bỏ từng thành phần:

Cấu hình	Val Accuracy	Chênh lệch
Pipeline đầy đủ	93.1%	—
(-) bỏ gradual unfreezing	91.8%	-1.3
(-) bỏ discriminative LR	92.4%	-0.7
(-) bỏ augmentation mạnh	89.5%	-3.6
(-) bỏ label smoothing	93.0%	-0.1

Đọc bảng: augmentation đóng góp lớn nhất (-3.6 khi bỏ đi); label smoothing gần như không ảnh hưởng (-0.1, nằm trong nhiễu) — có thể loại bỏ để pipeline gọn hơn. Đây là loại kết luận *chỉ có ablation mới cho được*.

Lưu ý quan trọng

Ablation study chỉ đáng tin khi *mỗi lần chỉ thay đổi một biến* và các yếu tố còn lại *được giữ cố định* (cùng seed, cùng split, cùng số epoch). Nếu không, bạn không thể quy kết chênh lệch cho thành phần nào. Đồng thời nên chạy mỗi cấu hình với vài seed để biết chênh lệch có vượt khỏi nhiễu hay không.

5.5 Error Analysis — Nhìn vào nơi mô hình sai

Khái niệm cốt lõi

Error analysis là việc *kiểm tra thủ công* các trường hợp mô hình dự đoán sai, nhằm tìm ra *mẫu hình* (pattern) trong lỗi — thay vì chỉ nhìn một con số accuracy tổng.

Vì sao cần: accuracy là một con số *tổng hợp* — nó che giấu *cấu trúc* của lỗi. Hai mô hình cùng đạt 90% có thể sai theo những cách hoàn toàn khác nhau, với hệ quả thực tế rất khác nhau.

Ví dụ minh họa

Vì sao **error analysis** là không thể bỏ qua — một ví dụ y khoa. Một mô hình phân loại ảnh X-quang đạt 95% accuracy — nghe rất tốt. Error analysis cho thấy: trong 5% sai, phần lớn là *bỏ sót ca bệnh* (false negative) ở nhóm bệnh nhân hiếm. Trong y tế, bỏ sót bệnh nguy hiểm hơn nhiều so với báo động giả. Con số 95% đã *che giấu* một lỗi nghiêm trọng. Chỉ khi nhìn vào *từng ca sai*, ta mới phát hiện và sửa được — ví dụ bằng cách điều chỉnh ngưỡng quyết định hoặc cân lại hàm mất mát theo lớp.

Các kỹ thuật error analysis cụ thể nên dạy:

- **Ma trận nhầm lẫn (confusion matrix).** Cho thấy *cặp lớp nào* hay bị lẫn với nhau — gợi ý lớp nào cần thêm dữ liệu hoặc đặc trưng phân biệt tốt hơn.
- **Phân tầng theo nhóm con (slice analysis).** Đo accuracy riêng cho từng nhóm: ảnh sáng / tối, vật thể lớn / nhỏ, văn bản ngắn / dài. Phát hiện mô hình kém ở nhóm nào.
- **Xem trực tiếp ảnh sai + Grad-CAM.** Phát hiện shortcut learning: mô hình đúng vì lý do *sai* (nhìn nền, nhìn watermark).
- **Phân tích theo độ tự tin.** Mô hình sai mà *rất tự tin* là dấu hiệu nguy hiểm (calibration kém).

Bài tập

Bài tập 4.1 (Apply). Thêm logging W&B hoặc TensorBoard vào Lab 1 hoặc Lab 2. Ghi tối thiểu: loss train/val, accuracy val, learning rate theo bước.

Bài tập 4.2 (Analyze). Thực hiện một ablation study với ít nhất ba thành phần trên pipeline Lab 2. Trình bày dưới dạng bảng giống mục 4.4 và nêu thành phần quan trọng nhất.

Bài tập 4.3 (Create). Thực hiện error analysis trên mô hình tốt nhất của bạn: vẽ confusion matrix, chọn 10 ca sai “tự tin nhất”, kiểm tra thủ công, và đề xuất *một* cải tiến cụ thể dựa trên những gì quan sát được.

Insight cần đạt

Tracking, ablation và error analysis cùng biến deep learning từ *nghệ thuật thử-sai* thành *kỹ thuật có kỷ luật*. Tracking cho ta dữ liệu để so sánh; ablation cho ta biết *cái gì* hiệu quả; error analysis cho ta biết *vì sao* mô hình sai và *phải sửa gì*. Bỏ qua ba bước này, mọi con số accuracy đều đáng ngờ.

6. Kỹ thuật Hiện đại — PEFT, LoRA & QLoRA

6.1 Câu hỏi mở đầu

Câu hỏi gợi mở

Một mô hình ngôn ngữ lớn như Llama-3 8B có 8 tỉ tham số. Full fine-tuning đòi hỏi lưu gradient và trạng thái optimizer cho *toàn bộ* 8 tỉ tham số đó — ước tính cần $\sim 60\text{--}80$ GB VRAM, vượt xa một GPU tiêu dùng. **Liệu có thực sự cần cập nhật cả 8 tỉ tham số** để mô hình thích nghi một tác vụ mới? Trực giác của bạn nói gì?

6.2 Vấn đề: Full Fine-tuning không còn khả thi

Ở kỷ nguyên mô hình nền tảng (foundation models), full fine-tuning gặp ba rào cản:

- **Bộ nhớ.** Optimizer Adam lưu hai trạng thái (moment bậc 1 và 2) cho mỗi tham số. Cùng với gradient và trọng số, tổng chi phí $\approx 4\times$ kích thước mô hình — bất khả thi với GPU phổ thông.
- **Lưu trữ.** Mỗi tác vụ tạo ra một bản sao đầy đủ của mô hình. Phục vụ 100 khách hàng nghĩa là lưu 100 mô hình 8B — hàng terabyte.
- **Nguy cơ catastrophic forgetting** trên toàn bộ tham số.

6.3 Ý tưởng nền tảng: cập nhật trọng số có “hạng thấp”

Khái niệm cốt lõi

Giả thuyết hạng thấp (low intrinsic rank). Khi thích nghi một mô hình lớn sang tác vụ mới, *lượng thay đổi cần thiết* ΔW của ma trận trọng số nằm trong một không gian con có *số chiều thấp* hơn rất nhiều. Nói cách khác: ma trận cập nhật ΔW có *hạng thực tế* (intrinsic rank) nhỏ.

Nếu giả thuyết này đúng, ta không cần học cả ma trận $\Delta W \in \mathbb{R}^{d \times k}$ ($d \cdot k$ tham số). Ta chỉ cần *phân tích nó thành tích hai ma trận gầy*.

6.4 LoRA — Low-Rank Adaptation

Khái niệm cốt lõi

LoRA. Thay vì cập nhật trực tiếp ma trận trọng số $W_0 \in \mathbb{R}^{d \times k}$, ta *đóng băng* W_0 và học một số hạng cộng thêm có hạng thấp:

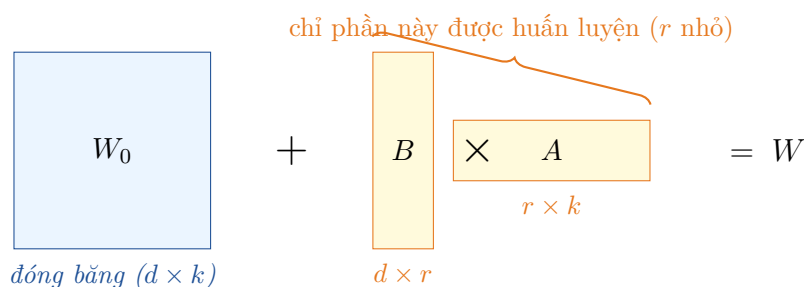
$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA,$$

trong đó $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, và **hạng** $r \ll \min(d, k)$. Chỉ A và B được huấn luyện; α là hệ số tỉ lệ (scaling).

Đếm tham số. Full fine-tuning của ma trận này cần $d \cdot k$ tham số. LoRA chỉ cần $r(d + k)$. Ví dụ với $d = k = 4096$ và $r = 8$:

$$\text{Full: } 4096 \times 4096 \approx 1.67 \times 10^7 \quad \text{vs} \quad \text{LoRA: } 8 \times (4096 + 4096) \approx 6.55 \times 10^4.$$

Giảm khoảng **256 lần** số tham số huấn luyện — chỉ cho *một* ma trận.



Insight cần đạt

Mấu chốt của LoRA: BA luôn là một ma trận *hạng tối đa bằng r* , bất kể A, B thế nào — vì hạng của tích không vượt quá hạng của thừa số nhỏ nhất. Ta *ép buộc* bản cập nhật phải có hạng thấp về mặt cấu trúc. Đây là cách biến giả thuyết “low intrinsic rank” thành một ràng buộc kiến trúc cụ thể.

Vài chi tiết kỹ thuật quan trọng:

- **Khởi tạo bất đối xứng — chi tiết sống còn.** A khởi tạo ngẫu nhiên (phân phối Gaussian), còn B khởi tạo *toàn số không*. Hệ quả: tại bước huấn luyện đầu tiên (step 0), $\Delta W = \frac{\alpha}{r} BA = \frac{\alpha}{r} \cdot \mathbf{0} \cdot A = \mathbf{0}$. Mô hình *xuất phát đúng bằng* mô hình nền pre-trained — không một chút nhiễu loạn nào.
- **Đặt ở đâu?** Thường áp dụng LoRA cho các ma trận chiếu trong attention (query, key, value, output). Đây là siêu tham số cần thử nghiệm.

Lưu ý quan trọng

Sai lầm khi tự cài đặt LoRA. Nếu sinh viên khởi tạo *cả* A và B một cách ngẫu nhiên (theo bản năng “khởi tạo trọng số thì phải random”), thì ngay tại step 0 ta đã có $\Delta W = \frac{\alpha}{r} BA \neq \mathbf{0}$ — một bản cập nhật *ngẫu nhiên vô nghĩa* bị cộng thẳng

vào trọng số gốc. Điều này **phá hủy tri thức pre-trained ngay từ epoch đầu tiên**, trước cả khi mô hình kịp học gì. Quy tắc phải nhớ: *một trong hai ma trận (B) bắt buộc khởi tạo bằng không* để đảm bảo $\Delta W = 0$ lúc bắt đầu. Đây là lý do vì sao khởi tạo của LoRA *bất đối xứng*.

Weight Merging — vì sao LoRA “miễn phí” khi suy luận

Trong lúc huấn luyện, LoRA giữ W_0 , A , B tách rời (vì chỉ A, B cần gradient). Nhưng khi triển khai thực tế (production / inference), ta thực hiện một bước **tái tham số hóa** (re-parameterization): cộng thẳng tích $\frac{\alpha}{r}BA$ vào trọng số gốc, tạo ra *một* ma trận duy nhất

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r}BA.$$

Sau bước gộp này, mô hình có *đúng* kiến trúc của mô hình nền — không còn nhánh A, B riêng, không còn phép nhân ma trận phụ. Mỗi lần forward, ta chỉ dùng W_{merged} y như một mô hình bình thường.

Insight cần đạt

Mấu chốt của LoRA gồm *hai* điều. **Thứ nhất**, BA luôn là ma trận hạng tối đa r , bất kể A, B — vì hạng của tích không vượt quá hạng của thừa số nhỏ nhất; ta đã ép bản cập nhật phải có hạng thấp về mặt cấu trúc. **Thứ hai**, nhờ weight merging, LoRA **không tốn thêm một mili-giây độ trễ (latency) nào** khi suy luận — khác hẳn Adapter (chèn module mới vào giữa mạng nên luôn có chi phí phụ). Chính đặc tính “rẻ khi huấn luyện và miễn phí khi triển khai” khiến LoRA trở thành lựa chọn số một của công nghiệp.

Ứng dụng thực tế

Hệ quả thực tế tuyệt vời: với mỗi tác vụ, ta chỉ cần lưu cặp (A, B) — thường vài MB thay vì hàng chục GB. Một mô hình nền 8B + 100 “adapter” LoRA cho 100 khách hàng vẫn gọn hơn *một* bản full fine-tune. Khi phục vụ, chỉ cần trao adapter tương ứng.

```

1  """
2  Minh hoa khai niem LoRA - Low-Rank Adaptation
3
4  Y tuong cot loi:
5  - Lop Linear goc co trong so W (shape: out x in), DONG BANG hoan toan
6
7  - Thay vi cap nhat W, ta hoc mot phan "delta" co rank thap: delta_W =
8    B @ A
9    + A co shape (r x in), khoi tao ngau nhien
10   + B co shape (out x r), khoi tao bang 0 -> luc dau delta_W = 0
11   + r << min(in, out) => RAT IT tham so can train
12 - Output: y = W @ x + (alpha / r) * (B @ A) @ x
13 - Chi A va B duoc train; W giu nguyen. Sau khi train co the "gop"
14   B @ A vao W de inference khong ton them chi phi.

```

```

13 """
14
15 import torch
16 import torch.nn as nn
17
18
19 # =====
20 # 1. LOP LINEAR CO GAN LORA
21 # =====
22 class LoRALinear(nn.Module):
23     """
24     Boc mot nn.Linear pretrained va them nhanh LoRA rank thap.
25     Trong so goc bi dong bang; chi ma tran A, B duoc hoc.
26     """
27
28     def __init__(self, base_linear: nn.Linear, r: int = 8, alpha: int
29                 = 16):
30         super().__init__()
31         self.base = base_linear                # lop Linear goc (
32             pretrained)
33         in_features = base_linear.in_features
34         out_features = base_linear.out_features
35
36         # --- Dong bang trong so goc ---
37         for param in self.base.parameters():
38             param.requires_grad = False
39
40         self.r = r
41         self.alpha = alpha
42         self.scaling = alpha / r                # he so ti le cho nhanh
43             LoRA
44
45         # --- Hai ma tran rank thap ---
46         # A: (r x in), khoi tao ngau nhien nho
47         self.lora_A = nn.Parameter(torch.randn(r, in_features) *
48                                     0.01)
49         # B: (out x r), khoi tao bang 0 -> ban dau delta_W = B @ A =
50             0
51         self.lora_B = nn.Parameter(torch.zeros(out_features, r))
52
53     def forward(self, x):
54         # Duong di goc (dong bang)
55         base_out = self.base(x)
56         # Nhanh LoRA: x -> A -> B, roi nhan he so ti le
57         # x: (... , in) ; lora_A.T: (in x r) ; lora_B.T: (r x out)
58         lora_out = (x @ self.lora_A.T @ self.lora_B.T) * self.scaling
59         return base_out + lora_out
60
61 @torch.no_grad()
62 def merge(self):
63     """Gop B @ A vao trong so goc -> inference khong ton them

```

```

        tinh_toan."""
59     delta_W = (self.lora_B @ self.lora_A) * self.scaling
60     self.base.weight.add_(delta_W)
61     # Sau khi gop, xoa dong gop cua nhanh LoRA de tranh tinh 2
        lan
62     self.lora_B.zero_()
63
64
65     # =====
66     # 2. HAM TIEN ICH: dem tham so train duoc
67     # =====
68     def count_params(model):
69         trainable = sum(p.numel() for p in model.parameters() if p.
            requires_grad)
70         total = sum(p.numel() for p in model.parameters())
71         return trainable, total
72
73
74     # =====
75     # 3. MO HINH VI DU: MLP nho, thay cac Linear bang LoRALinear
76     # =====
77     class SimpleMLP(nn.Module):
78         def __init__(self, dim=512, num_classes=10):
79             super().__init__()
80             self.fc1 = nn.Linear(dim, dim)
81             self.act = nn.ReLU()
82             self.fc2 = nn.Linear(dim, num_classes)
83
84         def forward(self, x):
85             return self.fc2(self.act(self.fc1(x)))
86
87
88     def apply_lora(model, r=8, alpha=16):
89         """Duyet model, thay moi nn.Linear bang LoRALinear boc no lai."""
90         for name, module in model.named_children():
91             if isinstance(module, nn.Linear):
92                 setattr(model, name, LoRALinear(module, r=r, alpha=alpha)
                    )
93             else:
94                 apply_lora(module, r=r, alpha=alpha) # de quy cho module
                    long nhau
95         return model
96
97
98     # =====
99     # 4. DEMO: so sanh so tham so & train thu
100    # =====
101    def demo():
102        device = "cuda" if torch.cuda.is_available() else "cpu"
103
104        # --- Mo hinh goc (full fine-tuning se train toan bo) ---

```

```

105 model = SimpleMLP(dim=512, num_classes=10).to(device)
106 tr, tot = count_params(model)
107 print(f"Truoc LoRA | train duoc: {tr:,.} / {tot:,.} (100% tham so
    )")
108
109 # --- Gan LoRA ---
110 model = apply_lora(model, r=8, alpha=16).to(device)
111 tr, tot = count_params(model)
112 print(f"Sau LoRA | train duoc: {tr:,.} / {tot:,.} "
113       f"({100*tr/tot:.2f}% tham so)")
114 print("=> Chi mot phan rat nho tham so can cap nhat.\n")
115
116 # --- Train thu chi cac tham so LoRA ---
117 optimizer = torch.optim.Adam(
118     filter(lambda p: p.requires_grad, model.parameters()), lr=1e
119     -3
120 )
121 criterion = nn.CrossEntropyLoss()
122
123 model.train()
124 for step in range(5):
125     x = torch.randn(32, 512, device=device)
126     y = torch.randint(0, 10, (32,), device=device)
127
128     optimizer.zero_grad()
129     loss = criterion(model(x), y)
130     loss.backward()
131     optimizer.step()
132     print(f" step {step+1}/5 - loss: {loss.item():.4f}")
133
134 # --- Gop trong so LoRA vao mo hinh goc ---
135 model.eval()
136 x_test = torch.randn(4, 512, device=device)
137 out_before = model(x_test) # output truoc khi gop
138
139 for module in model.modules():
140     if isinstance(module, LoRALinear):
141         module.merge()
142
143 out_after = model(x_test) # output sau khi gop
144 diff = (out_before - out_after).abs().max().item()
145 print(f"\nChenh lech output truoc/sau khi merge: {diff:.2e}")
146 print("=> Gan bang 0: merge khong lam doi ket qua, "
147       "nhung inference khong con tinh nhanh phu.")

```

6.5 QLoRA — Quantized LoRA

LoRA đã giảm tham số *huấn luyện*, nhưng mô hình nền W_0 vẫn phải nằm trong bộ nhớ ở độ chính xác 16-bit. Với mô hình 65B, riêng việc *chứa* W_0 đã quá tải GPU đơn. QLoRA giải quyết nốt phần này.

Khái niệm cốt lõi

QLoRA kết hợp LoRA với **lượng tử hóa** (quantization): mô hình nền W_0 được nén xuống **4-bit** và *đóng băng* ở dạng đó; chỉ các ma trận LoRA A, B được giữ ở độ chính xác cao và huấn luyện. Ba đóng góp kỹ thuật chính:

- **NF4 (4-bit NormalFloat)**. Một kiểu dữ liệu 4-bit được thiết kế tối ưu cho trọng số có phân phối chuẩn — giữ thông tin tốt hơn 4-bit thường.
- **Double quantization**. Lượng tử hóa cả các hằng số lượng tử hóa, tiết kiệm thêm bộ nhớ.
- **Paged optimizers**. Dùng bộ nhớ thống nhất CPU–GPU để xử lý các đỉnh bộ nhớ, tránh lỗi tràn (out-of-memory).

Insight cần đạt

Trực giác về vì sao QLoRA hoạt động: mô hình nền chỉ cần *đủ tốt để chạy xuôi* (forward pass) — một chút sai số lượng tử hóa 4-bit chấp nhận được. Còn *phần học* — các ma trận LoRA — vẫn ở độ chính xác cao, nên gradient vẫn sạch. QLoRA tách bạch: **phần cần chính xác cao thì giữ chính xác cao, phần chỉ cần chạy thì nén mạnh**. Kết quả: có thể fine-tune mô hình 65B trên *một* GPU 48 GB.

```

1      """
2  Minh hoa khai niem QLoRA - Quantized LoRA
3
4  QLoRA = Quantization (luong tu hoa) + LoRA. Y tuong:
5  - Van de cua LoRA: trong so goc W van phai nam trong bo nho o dang
    FP16/FP32
6  -> mo hinh lon (vai chuc ty tham so) van khong vua VRAM.
7  - Giai phap QLoRA:
8      1. NEN trong so goc W xuong 4-bit (NF4) -> giam ~4 lan bo nho.
9      2. Trong so 4-bit nay bi DONG BANG, chi dung de tinh forward.
10     3. Khi tinh toan, W duoc giai nen tam thoi ve bf16.
11     4. Nhanh LoRA (A, B) van o bf16 va la phan DUY NHAT duoc train.
12  - Ket qua: fine-tune mo hinh rat lon tren 1 GPU pho thong.
13  """
14
15  import torch
16  import torch.nn as nn
17
18
19  # =====
20  # 1. LUONG TU HOA 4-BIT (minh hoa don gian hoa)
21  # =====
22  # QLoRA thuc te dung NF4 (NormalFloat4) - kieu du lieu 4-bit toi uu
    cho
23  # trong so phan phoi chuan. O day ta minh hoa bang luong tu hoa 4-bit
24  # theo nhom (block-wise) de thay y tuong: chia W thanh cac block nho,
25  # moi block luu cac chi so 4-bit + mot he so scale rieng.

```

```

26
27 def quantize_4bit(weight: torch.Tensor, block_size: int = 64):
28     """Nen tensor float ve dang 4-bit theo tung block."""
29     flat = weight.flatten()
30     # Dem cho du chia het cho block_size
31     pad = (block_size - flat.numel() % block_size) % block_size
32     if pad:
33         flat = torch.cat([flat, torch.zeros(pad, device=flat.device)
34                             ])
35     blocks = flat.view(-1, block_size)
36
37     # Moi block: scale = max tri tuyet doi -> chuan hoa ve [-1, 1]
38     scales = blocks.abs().max(dim=1, keepdim=True).values
39     scales = torch.where(scales == 0, torch.ones_like(scales), scales)
40
41     normalized = blocks / scales
42
43     # Luong tu hoa ve 16 muc (4-bit): chi so nguyen 0..15
44     # Anh xa [-1, 1] -> [0, 15]
45     quant = torch.round((normalized + 1) * 7.5).clamp(0, 15).to(torch
46         .uint8)
47
48     return quant, scales, weight.shape, pad
49
50 def dequantize_4bit(quant, scales, original_shape, pad, block_size
51     =64):
52     """Giai nen tu 4-bit ve float (lam tam thoi khi tinh forward)."""
53     # Anh xa nguoc [0, 15] -> [-1, 1]
54     normalized = quant.float() / 7.5 - 1.0
55     blocks = normalized * scales
56     flat = blocks.flatten()
57     if pad:
58         flat = flat[:-pad]
59     return flat.view(original_shape)
60
61 # =====
62 # 2. LOP LINEAR KIEU QLORA
63 # =====
64 class QLoRALinear(nn.Module):
65     """
66     Linear voi trong so goc bi NEN 4-bit (dong bang) + nhanh LoRA
67     bf16.
68     Chi nhanh LoRA duoc train.
69     """
70
71     def __init__(self, base_linear: nn.Linear, r: int = 8, alpha: int
72         = 16,
73                 block_size: int = 64):
74         super().__init__()

```

```

71     in_f = base_linear.in_features
72     out_f = base_linear.out_features
73     self.in_features, self.out_features = in_f, out_f
74     self.block_size = block_size
75
76     # --- Luong tu hoa trong so goc xuong 4-bit ---
77     quant, scales, shape, pad = quantize_4bit(
78         base_linear.weight.data, block_size
79     )
80     # Luu duoi dang buffer: KHONG phai parameter -> khong train,
      khong co gradient
81     self.register_buffer("w_quant", quant)
82     self.register_buffer("w_scales", scales)
83     self.w_shape = shape
84     self.w_pad = pad
85
86     # Bias (neu co) giu nguyen, dong bang
87     if base_linear.bias is not None:
88         self.register_buffer("bias", base_linear.bias.data.clone())
89     else:
90         self.bias = None
91
92     # --- Nhanh LoRA (phan duy nhat duoc train) ---
93     self.r = r
94     self.scaling = alpha / r
95     self.lora_A = nn.Parameter(torch.randn(r, in_f) * 0.01)
96     self.lora_B = nn.Parameter(torch.zeros(out_f, r)) # B=0 ->
      luc dau delta=0
97
98     def forward(self, x):
99         # Giai nen trong so 4-bit ve float NGAY TRUOC khi tinh (khong
      luu lai)
100         w = dequantize_4bit(
101             self.w_quant, self.w_scales, self.w_shape, self.w_pad,
102             self.block_size
103         ).to(x.dtype)
104
105         # Duong di goc (trong so dong bang)
106         base_out = nn.functional.linear(x, w, self.bias)
107
108         # Nhanh LoRA bf16/fp32
109         lora_out = (x @ self.lora_A.T @ self.lora_B.T) * self.scaling
110         return base_out + lora_out
111
112     # =====
113     # 3. HAM TIEN ICH
114     # =====
115     def count_params(model):
116         trainable = sum(p.numel() for p in model.parameters() if p.

```

```

        requires_grad)
117     return trainable
118
119
120 def estimate_memory_mb(model):
121     """Uoc luong bo nho trong so: 4-bit cho w_quant, 32-bit cho phan
        con lai."""
122     bits = 0
123     for buf in model.buffers():
124         if buf.dtype == torch.uint8:           # trong so 4-bit luu trong
            uint8                                # moi gia tri thuc su chi
            bits += buf.numel() * 4              4-bit
125         else:
126             bits += buf.numel() * 32
127     for p in model.parameters():
128         bits += p.numel() * 32
129     return bits / 8 / 1024 / 1024
130
131
132
133 # =====
134 # 4. MO HINH VI DU
135 # =====
136 class SimpleMLP(nn.Module):
137     def __init__(self, dim=1024, num_classes=10):
138         super().__init__()
139         self.fc1 = nn.Linear(dim, dim)
140         self.act = nn.ReLU()
141         self.fc2 = nn.Linear(dim, num_classes)
142
143     def forward(self, x):
144         return self.fc2(self.act(self.fc1(x)))
145
146
147 def apply_qlora(model, r=8, alpha=16):
148     """Thay moi nn.Linear bang QLoRALinear."""
149     for name, module in model.named_children():
150         if isinstance(module, nn.Linear):
151             setattr(model, name, QLoRALinear(module, r=r, alpha=alpha
            ))
152         else:
153             apply_qlora(module, r=r, alpha=alpha)
154     return model
155
156
157 # =====
158 # 5. DEMO
159 # =====
160 def demo():
161     device = "cuda" if torch.cuda.is_available() else "cpu"
162

```



```

163 # --- Mo hình gốc FP32 ---
164 model_fp = SimpleMLP(dim=1024, num_classes=10).to(device)
165 mem_fp = estimate_memory_mb(model_fp)
166 print(f"Mo hình gốc FP32 | bộ nhớ trong số: {mem_fp:.2f} MB")
167
168 # --- Áp dụng QLoRA ---
169 model = SimpleMLP(dim=1024, num_classes=10)
170 model.load_state_dict(model_fp.state_dict()) # cùng trong số để
    so sánh
171 model = apply_qlora(model, r=8, alpha=16).to(device)
172
173 mem_q = estimate_memory_mb(model)
174 print(f"Mo hình QLoRA | bộ nhớ trong số: {mem_q:.2f} MB "
175       f"(giảm ~{mem_fp/mem_q:.1f} lần)")
176 print(f"Tham số train được: {count_params(model):,} "
177       f"(chi nhanh LoRA)\n")
178
179 # --- Train thử: chỉ nhanh LoRA có gradient ---
180 optimizer = torch.optim.Adam(
181     filter(lambda p: p.requires_grad, model.parameters()), lr=1e
182     -3
183 )
184 criterion = nn.CrossEntropyLoss()
185
186 model.train()
187 for step in range(5):
188     x = torch.randn(32, 1024, device=device)
189     y = torch.randint(0, 10, (32,), device=device)
190
191     optimizer.zero_grad()
192     loss = criterion(model(x), y)
193     loss.backward()
194     optimizer.step()
195     print(f"    step {step+1}/5 - loss: {loss.item():.4f}")
196
197 print("\n=> Trong số gốc 4-bit đồng bằng; chỉ A, B (bf16) được
    cap nhất.")

```

6.6 Họ kỹ thuật PEFT — bức tranh toàn cảnh

LoRA chỉ là một thành viên của họ **Parameter-Efficient Fine-Tuning (PEFT)**. Điều quan trọng cần nhận ra: các phương pháp PEFT chia thành *hai trường phái tư duy đối nghịch* về câu hỏi “thích nghi mô hình bằng cách nào”.

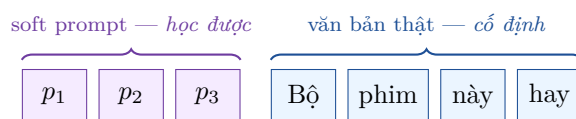
Phương pháp	Ý tưởng	Đặc điểm
LoRA / QLoRA	Cộng thêm bản cập nhật hạng thấp BA vào trọng số	Phổ biến nhất; gộp được, không chậm inference
Adapter	Chèn các module nhỏ (bottleneck) vào giữa các tầng	Thêm tham số và một chút độ trễ inference
Prefix-tuning	Học thêm các vector “tiền tố” gắn vào key/value của mọi tầng attention	Không đụng trọng số gốc
Prompt-tuning	Học các “soft prompt” — vài embedding ảo gắn vào đầu vào	Cực kỳ ít tham số; hiệu quả tăng theo cỡ mô hình

Hai trường phái: sửa trọng số vs thêm đầu vào ảo

Trường phái 1 — Can thiệp vào trọng số bên trong mạng. LoRA và Adapter đều sửa đổi cấu trúc tính toán của mô hình: LoRA cộng một nhánh hạng thấp vào ma trận trọng số; Adapter chèn hẳn module mới vào giữa các tầng. Tư duy ở đây: muốn mô hình hành xử khác, phải thay đổi bộ máy của nó.

Trường phái 2 — Không đụng trọng số, chỉ thêm “đầu vào ảo”. Đây là tư duy hoàn toàn khác của *Prompt-tuning* và *Prefix-tuning*:

- **Prompt-tuning.** Toàn bộ mô hình bị đóng băng tuyệt đối. Ta chỉ học thêm một số ít vector embedding — gọi là *soft prompt* hay *virtual tokens* — rồi gắn chúng vào đầu chuỗi đầu vào. Đây không phải từ ngữ thật (không tra được trong từ điển); chúng là các điểm tự do trong không gian embedding, được tối ưu bằng gradient descent để “điều hướng” mô hình về tác vụ mong muốn. Số tham số học cực nhỏ — chỉ vài nghìn đến vài chục nghìn.
- **Prefix-tuning.** Tổng quát hơn Prompt-tuning: thay vì chỉ thêm token ảo ở lớp đầu vào, ta thêm các vector “tiền tố” có thể huấn luyện vào key và value của attention ở mọi tầng. Nhờ can thiệp sâu hơn, nó thường mạnh hơn Prompt-tuning thuần túy, nhưng vẫn giữ nguyên tắc cốt lõi: *trọng số gốc bất khả xâm phạm*.



Insight cần đạt

Hai trường phái phản ánh hai câu trả lời cho cùng một câu hỏi. LoRA/Adapter: “thay đổi cỗ máy”. Prompt/Prefix-tuning: “giữ nguyên cỗ máy, chỉ thay đổi *thứ ta đưa vào*”. Một quan sát thú vị: Prompt-tuning gần như *không hiệu quả* với mô hình nhỏ, nhưng khi mô hình đủ lớn, nó tiệm cận hiệu năng của full fine-tuning — gợi ý rằng mô hình càng lớn thì “bề mặt hành vi” của nó càng có thể điều hướng chỉ bằng đầu vào, mà không cần đụng tới một tham số nào.

6.7 So sánh: Full Fine-tuning vs LoRA

Tiêu chí	Full Fine-tuning	LoRA
Tham số huấn luyện	100%	thường < 1%
Bộ nhớ GPU	Rất cao	Thấp
Lưu trữ mỗi tác vụ	Cả mô hình (GB)	Chỉ adapter (MB)
Tốc độ inference	Chuẩn	Chuẩn (sau khi gộp)
Hiệu năng đỉnh	Thường cao nhất	Rất gần, đôi khi ngang
Nguy cơ forgetting	Cao	Thấp (trọng số gốc giữ nguyên)

Lưu ý quan trọng

LoRA *không miễn phí*. Hiệu năng phụ thuộc mạnh vào hạng r và hệ số α : r quá nhỏ thì không đủ “dung lượng” để học tác vụ khó; r quá lớn thì mất đi lợi thế tiết kiệm và có thể overfitting. Với tác vụ *rất xa* dữ liệu pre-training, full fine-tuning vẫn có thể vượt trội. **Không có bữa trưa miễn phí** — luôn cần ablation theo r .

6.8 “Fine-tuning” trong kỷ nguyên LLM: ba cấp độ học

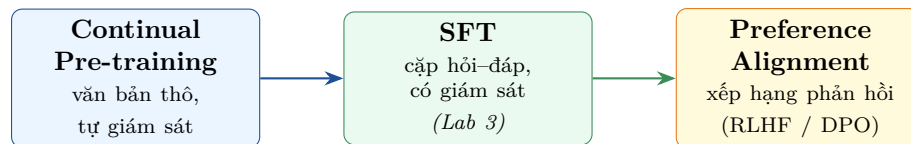
Trước khi vào Lab 3, cần làm rõ một điểm dễ gây nhầm lẫn. Với mô hình thị giác, “fine-tune” gần như chỉ có một nghĩa. Nhưng với LLM, từ “fine-tuning” bao trùm *ba* cấp độ học rất khác nhau về mục tiêu, loại dữ liệu, và vị trí trong quy trình:

Khái niệm cốt lõi

Cấp 1 — Continual Pre-training / Domain-Adaptive Pre-training. Huấn luyện *tự giám sát* (dự đoán token tiếp theo) tiếp tục trên một kho *văn bản thô* chuyên ngành — không nhãn. Mục tiêu: cho mô hình “ngấm” từ vựng, văn phong, kiến thức nền của một lĩnh vực (pháp lý, y khoa, tài chính). Đây thực chất là xử lý *domain shift* ở tầng ngôn ngữ.

Cấp 2 — Supervised Fine-Tuning (SFT). Huấn luyện *có giám sát* trên các cặp (chỉ thị / câu hỏi → câu trả lời mong muốn). Mục tiêu: dạy mô hình *tuân thủ chỉ thị* (instruction-following) và trả lời theo định dạng ta muốn. Lab 3 của module này nằm ở cấp độ này.

Cấp 3 — Preference Alignment (RLHF / DPO). Căn chỉnh mô hình theo *sở thích con người*: với cùng một câu hỏi, con người xếp hạng các câu trả lời, và mô hình học để ưu tiên kiểu trả lời được đánh giá cao — hữu ích, trung thực, an toàn. RLHF dùng học tăng cường; DPO là một phương pháp gọn hơn đạt mục tiêu tương tự mà không cần mô hình phần thưởng riêng.



Lab 3 tập trung vào đây

Lưu ý quan trọng

Cả ba cấp độ đều *có thể* dùng LoRA/QLoRA để tiết kiệm tài nguyên — PEFT không bị giới hạn ở một cấp nào. Điều sinh viên cần tránh là gọi chung chung “fine-tune một LLM” mà không nói rõ cấp độ: một mô hình đã qua SFT nhưng *chưa* alignment có thể trả lời đúng định dạng nhưng vẫn thiếu an toàn; ngược lại, continual pre-training mà không có SFT thì mô hình “biết nhiều” nhưng không biết *tuân lệnh*. Mỗi cấp giải một bài toán riêng.

Câu hỏi gợi mở

Một công ty luật muốn có trợ lý AI trả lời câu hỏi pháp lý Việt Nam. Họ có: (a) hàng GB văn bản luật thô, và (b) khoảng 2000 cặp câu hỏi–trả lời do luật sư soạn. Theo bạn, nên áp dụng cấp độ nào, theo *thứ tự* nào, và vì sao? Cấp độ nào là *bắt buộc*, cấp độ nào là *nên có*?

6.9 Code mẫu & Giới thiệu Lab 3

Listing 6: Fine-tune một LLM với LoRA bằng thư viện PEFT của Hugging Face

```

1 from transformers import AutoModelForCausalLM, AutoTokenizer,
  BitsAndBytesConfig
2 from peft import LoraConfig, get_peft_model
3 import torch
4
5 # 1. (QLoRA) Cấu hình nạp mô hình nén ở 4-bit NF4
6 bnb_config = BitsAndBytesConfig(
7     load_in_4bit=True,
8     bnb_4bit_quant_type="nf4",
9     bnb_4bit_compute_dtype=torch.bfloat16,
10    bnb_4bit_use_double_quant=True,
11 )
12 model = AutoModelForCausalLM.from_pretrained(
13     "meta-llama/Meta-Llama-3-8B", quantization_config=bnb_config,
14 )
15
16 # 2. Cấu hình LoRA: chỉ áp dụng cho các ma trận chiều trong attention
17 lora_config = LoraConfig(
18     r=16, lora_alpha=32, lora_dropout=0.05,
19     target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
20     task_type="CAUSAL_LM",
21 )
22 model = get_peft_model(model, lora_config)
23
  
```

```

24 # 3. Quan sát: chỉ một phần rất nhỏ tham số được huấn luyện
25 model.print_trainable_parameters()
26 # Ví dụ in ra: trainable params: 6.8M || all params: 8.0B ||
    trainable%: 0.085

```

Bài tập

Lab 3 — Fine-tune Llama-3 (hoặc Mistral) với LoRA / QLoRA.

Yêu cầu thực hiện:

1. Chọn một tác vụ instruction-following hoặc phân loại văn bản trên một bộ dữ liệu nhỏ.
2. Fine-tune bằng QLoRA. Ghi lại bộ nhớ GPU đỉnh và so với ước tính chi phí của full fine-tuning.
3. Ablation theo hạng $r \in \{4, 8, 16, 32, 64\}$: vẽ đồ thị hiệu năng theo r và bộ nhớ theo r .
4. Gộp (merge) adapter vào mô hình nền và xác nhận tốc độ inference không đổi.

Câu hỏi phân tích bắt buộc:

- Hiệu năng có *bão hòa* theo r không? Từ giá trị r nào trở đi việc tăng thêm không còn đáng giá?
- Với tác vụ của bạn, “điểm ngọt” (sweet spot) của r là bao nhiêu? Bảo vệ lựa chọn bằng số liệu.
- Khi nào bạn sẽ khuyến nghị full fine-tuning thay vì LoRA, dù LoRA rẻ hơn?

Insight cần đạt

PEFT đảo ngược một giả định cũ: “muốn mô hình học tác vụ mới thì phải cập nhật mô hình”. LoRA/QLoRA cho thấy ta chỉ cần học một *bản hiệu chỉnh hạng thấp* đặt cạnh mô hình đóng băng. Điều này dân chủ hóa việc fine-tuning LLM — biến nó từ đặc quyền của các phòng lab có hàng trăm GPU thành việc khả thi trên một GPU đơn.

7. Ứng dụng & Các Tình huống Thực tế

7.1 Câu hỏi mở đầu

Câu hỏi gợi mở

Mọi kỹ thuật ở các section trước đều giả định một điều: tồn tại một mô hình pre-trained “đủ gần” với bài toán của bạn. Trong thực tế, khi nào giả định đó *đúng*, và khi nào nó *sụp đổ*? Hãy nghĩ tới một bài toán mà Transfer Learning từ ImageNet sẽ *thất bại*.

7.2 Computer Vision

Ứng dụng thực tế

Ảnh y khoa (Medical Imaging). Phân loại X-quang, CT, MRI; phát hiện khối u. Thách thức: ảnh y khoa là ảnh xám, kết cấu rất khác ảnh tự nhiên; dữ liệu có nhãn cực hiếm và đắt (cần bác sĩ chuyên khoa). Hướng tiếp cận: pre-train trên ImageNet rồi fine-tune, hoặc tốt hơn là pre-train tự giám sát (self-supervised) trên kho ảnh y khoa *không nhãn* rồi mới fine-tune.

Ảnh vệ tinh (Satellite / Remote Sensing). Phân loại sử dụng đất, theo dõi phá rừng, ước lượng mùa màng. Thách thức: ảnh đa phổ (nhiều hơn 3 kênh RGB), độ phân giải và góc nhìn đặc thù. Thường cần điều chỉnh tầng đầu vào để nhận thêm kênh phổ.

Phát hiện lỗi sản phẩm (Defect Detection). Kiểm tra chất lượng trong sản xuất. Thách thức: dữ liệu *mất cân bằng* nghiêm trọng — sản phẩm lỗi rất hiếm. Transfer Learning giúp khởi đầu tốt; cần kết hợp kỹ thuật xử lý mất cân bằng và phát hiện bất thường (anomaly detection).

7.3 Xử lý Ngôn ngữ Tự nhiên (NLP)

Ứng dụng thực tế

Lĩnh vực chuyên biệt — pháp lý, y khoa tiếng Việt. Văn bản pháp lý và y khoa có từ vựng, cú pháp, và quy ước riêng mà mô hình huấn luyện trên văn bản phổ thông không nắm được. Hướng tiếp cận điển hình: lấy một mô hình tiếng Việt (ví dụ PhoBERT), *tiền huấn luyện thích nghi miền* (domain-adaptive pre-training) trên kho văn bản chuyên ngành không nhãn, rồi mới fine-tune cho tác vụ cụ thể (phân loại điều khoản, trích xuất thực thể y khoa).

Bài học chung: khi miền đích đặc thù, một bước *pre-training trung gian* trên dữ liệu *không nhãn cùng miền* thường mang lại lợi ích lớn — nó thu hẹp khoảng cách miền *trước khi* tác vụ giám sát bắt đầu.

7.4 Đa phương thức (Multimodal)

Khái niệm cốt lõi

CLIP học cách *căn chỉnh* (align) ảnh và văn bản vào *cùng một không gian embedding*, bằng cách huấn luyện trên hàng trăm triệu cặp (ảnh, chú thích) từ web. Kết quả: mô hình có thể phân loại *zero-shot* — nhận diện một lớp *chưa từng thấy nhãn* chỉ bằng cách so khớp ảnh với mô tả văn bản của lớp đó.

LLaVA ghép một bộ mã hóa thị giác (kiểu CLIP) với một LLM, tạo ra trợ lý có thể “nhìn” ảnh và trả lời câu hỏi về ảnh. Đây là Transfer Learning ở tầm *kết hợp hai mô hình nền* từ hai phương thức khác nhau.

Insight cần đạt

Multimodal là biên giới mới của Transfer Learning: thay vì chuyển tri thức *trong cùng một phương thức*, ta chuyển và *căn chỉnh* tri thức *giữa các phương thức*. Khả năng zero-shot của CLIP là minh chứng mạnh nhất cho luận điểm Section 1: một biểu diễn đủ tốt thì *tái sử dụng được* cho cả những tác vụ chưa từng thấy.

7.5 Gợi ý Guest Lecture**Ghi chú giảng viên**

Nếu điều kiện cho phép, mời một kỹ sư từ doanh nghiệp chia sẻ một case thực tế. Nên định hướng phần chia sẻ vào những điều *không có trong sách*: pipeline dữ liệu thực tế bản ra sao, quyết định chọn Feature Extraction hay Fine-tuning dựa trên ràng buộc ngân sách nào, các thất bại đã gặp và cách khắc phục. Sinh viên học nhiều nhất từ những câu chuyện thất bại được kể trung thực.

8. Đạo đức & Thiên kiến trong Mô hình Pre-trained**Câu hỏi gợi mở**

Khi bạn fine-tune một mô hình pre-trained, bạn *thừa hưởng* điều gì ngoài “đặc trưng hữu ích”? Nếu kho dữ liệu pre-training chứa thiên kiến xã hội, chuyện gì xảy ra với mô hình đích của bạn?

Lưu ý quan trọng

Transfer Learning chuyển giao *toàn bộ* những gì mô hình nền đã học — bao gồm cả **thiên kiến** (bias). Một mô hình pre-train trên dữ liệu web sẽ hấp thụ các định kiến về giới, chủng tộc, vùng miền có trong dữ liệu đó. Khi bạn fine-tune, các thiên kiến này *không tự biến mất*; chúng âm thầm đi vào sản phẩm của bạn. Đây là vấn đề *đạo đức lẫn kỹ thuật*, không phải tùy chọn.

Trách nhiệm tối thiểu của người làm Transfer Learning:

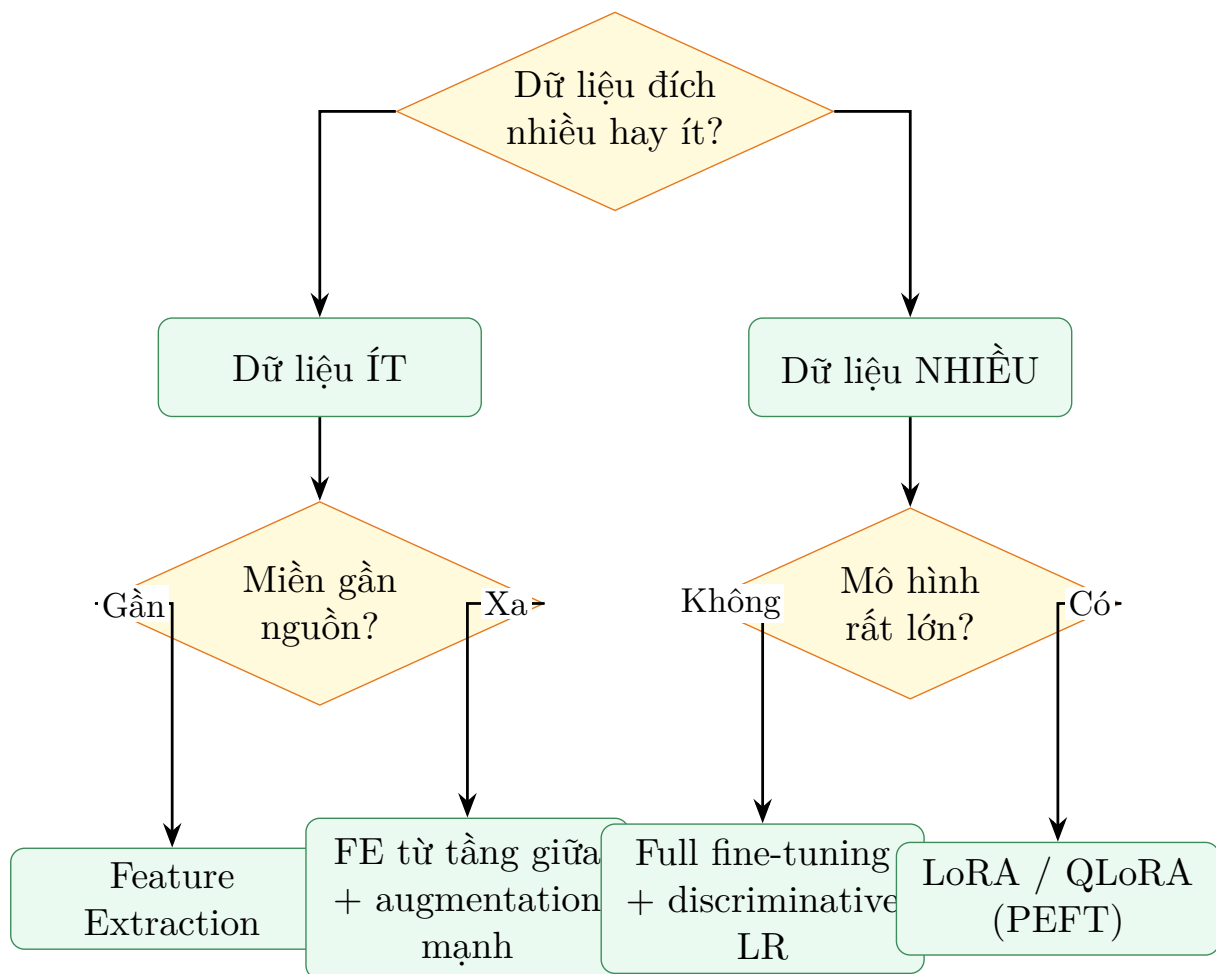
- **Biết nguồn gốc.** Tìm hiểu mô hình nền được huấn luyện trên dữ liệu nào; đọc model card nếu có.
- **Đánh giá phân tầng.** Đừng chỉ báo cáo accuracy tổng — đo hiệu năng riêng cho từng nhóm nhân khẩu / nhóm con (gắn với slice analysis ở Section 4).
- **Minh bạch.** Ghi rõ các hạn chế và thiên kiến đã biết của mô hình khi bàn giao.
- **Cân nhắc tác hại.** Trong các lĩnh vực nhạy cảm (y tế, tuyển dụng, tín dụng, tư pháp), một mô hình thiên lệch có thể gây hại thực sự cho con người.

Insight cần đạt

Quy tắc nghề nghiệp: hiệu năng cao *không* biện minh cho việc bỏ qua công bằng. Một mô hình 95% accuracy nhưng phân biệt đối xử có hệ thống với một nhóm người là một sản phẩm *hỏng*, bất kể con số đẹp đến đâu.

9. Tổng kết & Dự án Cuối kỳ

9.1 Sơ đồ ra quyết định



9.2 Những điều cốt lõi cần nhớ

- Transfer Learning là *thuật ngữ ô*; Feature Extraction và Fine-tuning là hai chiến lược bên trong nó.
- Nó hoạt động nhờ *tính phân cấp* của biểu diễn và vai trò *prior* của trọng số pre-trained.
- Chọn chiến lược dựa trên *lượng dữ liệu × khoảng cách miền* — không theo cảm tính.
- Fine-tuning đúng cần kỷ luật: làm ấm head, mở băng dần, LR nhỏ cho tầng nông.

- Tracking + ablation + error analysis biến thử-sai thành khoa học.
- PEFT (LoRA/QLoRA) dân chủ hóa fine-tuning cho mô hình cực lớn nhờ ý tưởng *cập nhật hạng thấp*.
- Transfer Learning thừa hưởng cả thiên kiến — trách nhiệm đạo đức là bắt buộc.

9.3 Dự án cuối kỳ (Create)

Bài tập

Dự án tổng hợp. Chọn một bài toán *domain-specific* thực tế (tự đề xuất hoặc từ danh sách của giảng viên). Yêu cầu:

1. **Phân tích.** Xác định domain shift / task shift; định vị bài toán trên ma trận dữ liệu $\times$ khoảng cách miền; lập luận chọn chiến lược.
2. **Triển khai.** Cài đặt ít nhất hai chiến lược để so sánh (ví dụ Feature Extraction vs LoRA).
3. **Kỷ luật thực nghiệm.** Dùng experiment tracking; thực hiện một ablation study và một error analysis đầy đủ.
4. **Sáng tạo.** Đề xuất *một* cải tiến không có trong bài giảng, dựa trên error analysis của chính bạn, và kiểm chứng nó.
5. **Đạo đức.** Một mục bàn về thiên kiến tiềm tàng và đánh giá phân tầng.
6. **Báo cáo.** Trình bày kết quả với bảng, biểu đồ, và lập luận rõ ràng.

Tiêu chí chấm: không chấm theo accuracy tuyệt đối, mà theo *chất lượng lập luận*, tính chặt chẽ của thực nghiệm, và chiều sâu của phân tích.

Tài liệu Tham khảo & Nguồn học

Giáo trình & khóa học chính

- *Hands-On Transfer Learning with Python* — giáo trình nền tảng của module.
- Stanford CS231n — *Convolutional Neural Networks for Visual Recognition* (phần Transfer Learning).
- Hugging Face Course — phần fine-tuning Transformer và thư viện PEFT.

Kho mã & lab thực hành

- `mrdbourne/pytorch-deep-learning` — chương 06 về Transfer Learning.
- Các ví dụ chính thức của thư viện Hugging Face PEFT.

Bài báo nền tảng (đọc thêm)

- Yosinski et al. — “How transferable are features in deep neural networks?”
- Hu et al. — “LoRA: Low-Rank Adaptation of Large Language Models.”
- Dettmers et al. — “QLoRA: Efficient Finetuning of Quantized LLMs.”
- Radford et al. — “Learning Transferable Visual Models from Natural Language Supervision” (CLIP).

Hỗ trợ tiếng Việt

- Loạt bài viết của Phạm Đình Khánh về Transfer Learning và fine-tuning.

“Standing on the shoulders of giants.”

— Tinh thần cốt lõi của Transfer Learning —